

Chiplet Design Automation: Methodologies, Advances, and Directions

FUPING LI, SKLP, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

DING YUAN, CICS, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

JIXIANG ZHU, CICS, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

XIHAO LIANG, CICS, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

JUELEI ZHOU, CICS, Institute of Computing Technology, CAS, Beijing, China

TIAN MIAO, CICS, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

HUI LI, Jinan Inspur Data Technology Co., Ltd., Jinan, China

KAI ZHANG, Jinan Inspur Data Technology Co., Ltd., Jinan, China

YINHE HAN, CICS, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

HUAWEI LI, SKLP, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

This work was supported by the National Natural Science Foundation of China (Grant No. 92373205, 92373206, 62222411, and 62025404) and the National Key Research and Development Program of China (Grant No. 2023YFB4404400).

Authors' Contact Information: Fuping Li, SKLP, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: lifuping20s@ict.ac.cn; Ding Yuan, CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: yuanding23s@ict.ac.cn; Jixiang Zhu, CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: zhujixiang231@mailsucas.ac.cn; Xihao Liang, CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: liangxihao22@mailsucas.ac.cn; Juele Zhou, CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China; e-mail: zjl199904@mail.ustc.edu.cn; Tian Miao, CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: miaotian22@mailsucas.ac.cn; Hui Li, Jinan Inspur Data Technology Co., Ltd., Jinan, China; e-mail: hui.lee@inspur.com; Kai Zhang, Jinan Inspur Data Technology Co., Ltd., Jinan, China; e-mail: zhangkai_bj@inspur.com; Yinhe Han, CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: yinhes@ict.ac.cn; Huawei Li, SKLP, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: lihuawei@ict.ac.cn; Xiaowei Li, SKLP, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: lxw@ict.ac.cn; Ying Wang (corresponding author), CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: wangying2009@ict.ac.cn; Yujie Wang (corresponding author), CICS, Institute of Computing Technology, CAS, Beijing, Beijing, China and University of Chinese Academy of Sciences, Beijing, Beijing, China; e-mail: wangyujie@ict.ac.cn.



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/).

© 2026 Copyright held by the owner/author(s).

ACM 1084-4309/2026/04-ART85

<https://doi.org/10.1145/3796529>

XIAOWEI LI, SKLP, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

YING WANG, CICS, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

YUJIE WANG, CICS, Institute of Computing Technology, CAS, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

With the slowdown of Moore's Law, conventional monolithic chip architectures face challenges such as excessive die sizes and prohibitive manufacturing costs. Consequently, chiplets have emerged as a pivotal technology in the post-Moore era, attracting significant attention from both academia and industry. Multi-chiplet systems offer compelling advantages over monolithic ones, including enhanced integration density, reduced cost, and shortened time-to-market. However, realizing these benefits necessitates design flows capable of optimizing parameters across logical, physical, and circuit layers, which introduces substantial design complexity. Numerous design automation technologies have been proposed to address these challenges. This article provides a comprehensive overview of related advancements, categorizing chiplet design methodologies into two primary types: (i) **top-down** flows disintegrating existing hardware designs into chiplets and subsequently reintegrating them into multi-chiplet systems, and (ii) **bottom-up** flows combining existing chiplets into multi-chiplet systems based on user applications. This article begins by introducing foundational concepts, technical characteristics, and evaluation models relevant to multi-chiplet systems. We then systematically summarize the problem formulations, design spaces, and optimization techniques associated with top-down and bottom-up design flows. Finally, we discuss key challenges and potential future research directions in chiplet design automation, aimed at further harnessing the potential of chiplet-based integration.

CCS Concepts: • **Hardware** → **Methodologies for EDA**; **Integrated circuits**; • **General and reference** → **Surveys and overviews**;

Additional Key Words and Phrases: Chiplet, advanced packaging, electronic design automation, design methodology

ACM Reference Format:

Fuping Li, Ding Yuan, Jixiang Zhu, Xihao Liang, Juelei Zhou, Tian Miao, Hui Li, Kai Zhang, Yinhe Han, Huawei Li, Xiaowei Li, Ying Wang, and Yujie Wang. 2026. Chiplet Design Automation: Methodologies, Advances, and Directions. *ACM Trans. Des. Autom. Electron. Syst.* 31, 5, Article 85 (April 2026), 56 pages. <https://doi.org/10.1145/3796529>

1 Introduction

For decades, Moore's Law has been the key enabler of the semiconductor industry. However, as depicted in Figure 1, transistor scaling becomes more difficult as dimensions approach atomic levels. Consequently, research and development costs are skyrocketing for both foundries pioneering new generations of process nodes and design houses employing them [112]. Moreover, emerging applications like **machine learning (ML)** continually drive up the demand for chip computational power. The gap between the slowdown in transistor scaling and escalating computational demands continues to widen. This trend results in ever-increasing die sizes, which suffer from reduced manufacturing yields and can become unmanufacturable when they exceed the reticle limit [143].

To address this dilemma, chiplet technology has emerged as a pivotal More-than-Moore solution [166]. Essentially, chiplets are physical **intellectual property (IP)** blocks with standardized

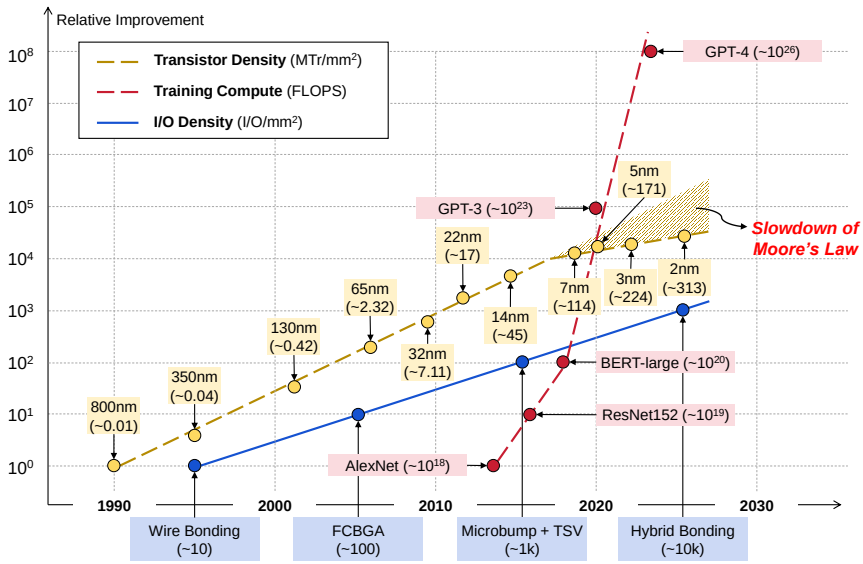


Fig. 1. Growing trend of transistor density, training compute for ML models, and I/O density [172, 190].

interfaces that are integrated via packages to form synergistic multi-chiplet systems. Although the chiplet concept dates back to IBM's **multi-chip module (MCM)** research in the 1980s [197], the industry primarily focused on scaling monolithic **systems-on-chips (SoCs)** for a long period. However, a significant shift is now underway, driven by breakthroughs in advanced packaging. Figure 1 illustrates that the chiplet I/O density has grown exponentially in recent years, contrasting with the slowing pace of transistor scaling. This enables the integration of multiple small, high-yield chiplets using low-overhead interconnects. As a result, it is now feasible to create efficient multi-chiplet systems with computational capabilities surpassing the reticle limit, addressing the area, yield, and cost challenges of monolithic architectures. Furthermore, flexible chiplet decomposition and combination significantly enhance design reuse and shorten the development cycle. This technology is now seeing widespread commercial adoption, with numerous chiplet-based products launched by industry leaders like AMD [143, 181], Intel [53, 58], and HiSilicon [214].

1.1 Our Taxonomy of Chiplet Design Methodologies

Chiplet technology is revolutionizing the chip design process by transitioning from the traditional two-level IP-chip hierarchy to a three-level *IP-chiplet-chip* hierarchy. Based on their design starting points, current chiplet design methodologies can be classified into two orthogonal types:

- **Type I: Top-down or decomposition-based flow.** As shown in Figure 2(a), this approach involves disintegrating one or more *existing hardware designs* into multiple smaller chiplets [86, 96, 111]. These chiplets are then reintegrated using packaging technologies to construct multi-chiplet systems that are functionally equivalent to the original designs, offering benefits such as improved manufacturing yields.
- **Type II: Bottom-up or combination-based flow.** As depicted in Figure 2(b), this approach refers to selecting and integrating *existing chiplets* within a package to form multi-chiplet systems tailored to specific applications [108, 114]. Its main advantages are the significantly shortened development cycle and the efficient reuse of verified chiplets.

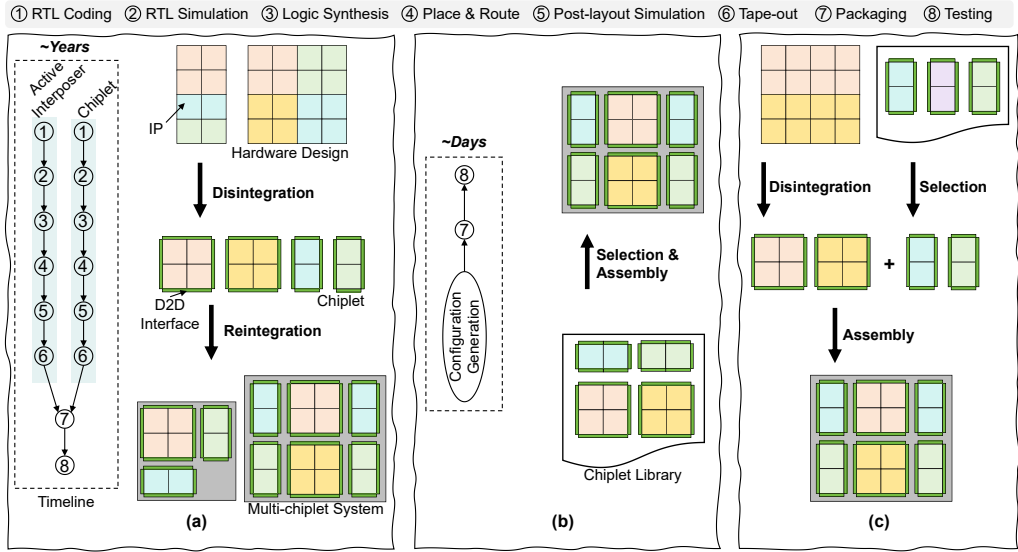


Fig. 2. Overview of different chiplet design methodologies: (a) top-down, (b) bottom-up, and (c) hybrid.

The primary distinctions between these two methodologies include:

(1) **Design goal:** The top-down approach aims at meeting highly customized functional and performance requirements. In contrast, the bottom-up approach assembles the target chip using off-the-shelf chiplets, with performance and overhead constrained by available chiplets.

(2) **Design flexibility:** The top-down approach offers high flexibility in tuning design parameters of chiplets and packages. Conversely, because the bottom-up approach utilizes existing chiplets with pre-defined functionalities, its design space is inherently constrained.

(3) **Development cycle:** The bottom-up approach can significantly shorten a product's time-to-market to days by leveraging verified chiplets and reusable packages. While the top-down approach can be accelerated by chiplet reuse, it still requires considerably more time for conventional tape-out stages such as RTL design, verification, and physical implementation.

(4) **Reliance on chiplet ecosystem:** The bottom-up method relies heavily on a mature chiplet ecosystem featuring standardized **die-to-die (D2D)** interfaces and robust interoperability. In contrast, the top-down approach using in-house chiplets is less dependent on this ecosystem.

These two methodologies can be further fused to leverage their respective strengths. As exemplified in Figure 2(c), chiplets derived from top-down decomposition for performance-critical components can be combined with third-party chiplets via a bottom-up flow. A prominent industry example of this hybrid approach is AMD Instinct MI300X [179] accelerator, which combines custom core compute and I/O chiplets with HBM chiplets sourced from third-party vendors. This balances performance, cost, and development cycle. Regardless of the chosen methodology, designers must manage numerous chiplet and packaging parameters, which constitute a vast and intricate design space. Manual efforts alone are often insufficient for identifying globally optimal solutions within this space, thereby posing significant challenges and new opportunities for the **electronic design automation (EDA)** community. Focusing on these two core design methodologies, our article presents a comprehensive survey of design automation technologies for multi-chiplet systems.

Table 1. Summary of Surveys on Chiplet Technology

Surveys	Primary Focus	Methodology Taxonomy	Design Methodology Analysis	
			Top-down	Bottom-up
[112]	Concepts and Tech.	×	×	×
[132]	Interfaces and Interconnects	×	×	×
[67]	Hardware Systems	×	×	×
[110]	Manual Design Exp.	✓	✓	✓
[22]	Narrow EDA (2.5D ICs)	×	✓	×
[128]	Narrow EDA (3D ICs)	×	✓	×
This Work	EDA	✓✓	✓✓	✓✓

Legend: ✓✓ (Comprehensive Coverage), ✓ (Covered), × (Not Covered).

1.2 Major Differences with Existing Surveys

While several surveys on chiplet technology exist, they lack a comprehensive, methodology-driven view of design automation. For instance, Li et al. [112] and Ma et al. [132] elucidate fundamental concepts like D2D interfaces and packaging technologies, but they offer limited discussion on design automation. The recent and comprehensive survey [67] provides an excellent overview of the system-level architectures and physical implementation challenges of large-scale integration, but it does not delve into the specific EDA methodologies that enable such complex designs. Similarly, the study [110] summarizes experiences in manual chiplet design but omits automation aspects. The few available design automation surveys [22, 128] have narrow scopes, primarily focusing on top-down design flows specific to particular packaging technologies (e.g., 2.5D or 3D integration) and neglecting the bottom-up approach. To clearly position our contribution against this backdrop, we present a detailed comparison in Table 1.

To fill this gap, our article makes three major **contributions**. First, it establishes a foundation by reviewing the concepts, technical characteristics, and evaluation models relevant to multi-chiplet systems. Second, it presents a user-centric and methodology-driven perspective that gives equal prominence to both top-down and bottom-up design flows, formally defining their associated design stages and optimization techniques in the context of mainstream packaging technologies. Finally, it identifies key challenges and future research directions in chiplet design automation to further enhance development efficiency and reduce costs.

1.3 Survey Outline

In this article, Section 2 reviews the fundamentals of chiplet technology, including architecture, tradeoffs, and evaluation models. Sections 3 and 4 then detail the problem formulations, design spaces, and optimization techniques for top-down and bottom-up design flows, respectively. Finally, Section 5 explores key challenges and future research directions for creating a unified and efficient chiplet design platform.

2 Landscape of Chiplet Technology

This section begins with a foundational overview of chiplet technology by detailing its core components. Subsequently, it discusses the advantages and limitations of this approach. Finally, the evaluation models that are essential for design automation are introduced.

2.1 Multi-chiplet System Architecture

Figure 3 shows a typical multi-chiplet system with chiplets stacked horizontally or vertically on an interposer. The inter-router links established by D2D interfaces, **through-silicon vias**

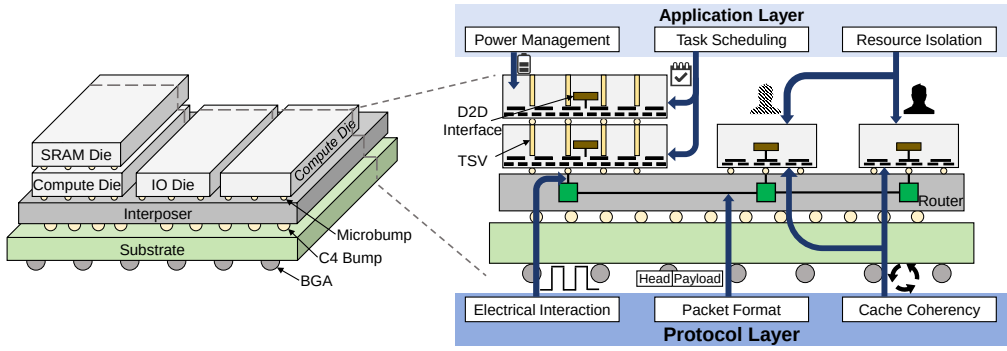


Fig. 3. The architecture of a multi-chiplet system.

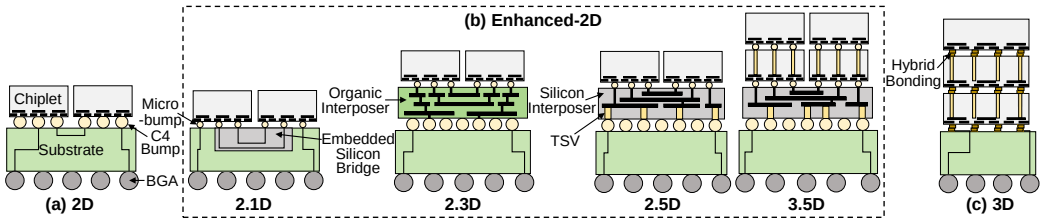


Fig. 4. An overview of advanced packaging technologies, categorized as: (a) 2D, (b) enhanced-2D, and (c) 3D.

(TSVs), and microbumps create an inter-chiplet network. Underlying protocols define key aspects such as electrical interactions, network packet formats, and cache coherence [173]. Building on this foundation, high-level applications execute functions like power management [113], task scheduling [7], and multi-tenant resource isolation [222]. Packaging technology, D2D interfaces, and inter-chiplet networks are foundational to chiplet design automation and are elaborated below.

A packaging technology is characterized by its materials and the spatial arrangement of chiplets, which can be horizontal or vertical. Following [2], we classify these technologies into three categories: 2D, enhanced-2D, and 3D, as illustrated in Figure 4. 2D packaging places chiplets side-by-side on an organic substrate, whereas 3D packaging stacks them vertically. Enhanced-2D packaging encompasses various technologies falling between pure 2D and 3D [102]. Examples include 2.1D utilizing embedded silicon bridges, 2.3D employing high-density organic interposers, 2.5D based on silicon or glass interposers, and 3.5D horizontally connecting multiple 3D stacks. These options represent different tradeoffs in cost, interconnect density, thermal dissipation, and scalability [199]. Active silicon interposers, for example, can utilize repeaters to optimize communication. However, they are more expensive than passive interposers due to complex **front-end-of-line (FEOL)** processes [29]. Silicon bridges offer high bandwidth between adjacent chiplets and are more cost-effective than larger-area silicon interposers [36]. In comparison, non-adjacent chiplets must communicate via intermediate chiplets, potentially increasing power consumption and latency.

The long horizontal interconnects in 2D and enhanced-2D systems necessitate D2D interfaces to improve drive strength and ensure signal integrity. These interfaces are typically classified as serial or parallel [132]. Serial interfaces, such as USR [12], use encoding and serialization to support longer transmission distances with fewer I/O connections. While this makes them suitable for low-density organic substrates, it comes at the cost of higher power consumption and latency. In contrast,

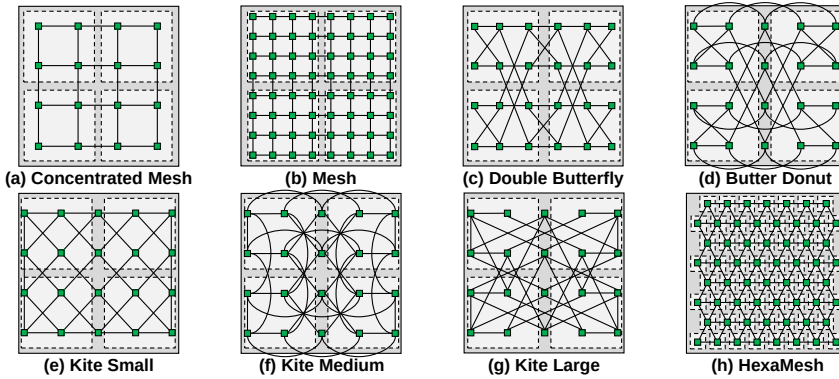


Fig. 5. Examples of common inter-chiplet network topologies, where dotted boxes represent chiplets and green boxes denote routers.

parallel interfaces, like AIB [91], transmit data directly and are better suited for high-density silicon interposers. To leverage the strengths of both, Feng et al. [47] propose a hybrid approach that integrates serial and parallel interfaces within a single system. In 3D systems, the extremely short and fine-pitch vertical interconnects enable PHY-less interfaces driven by simple digital logic [78]. This helps to overcome the bandwidth bottlenecks and energy efficiency losses inherent to planar routing, improving throughput and communication efficiency, especially for DNN accelerators [141]. Nevertheless, with the exception of face-to-face bonding, these systems rely on TSVs for inter-chiplet communication, which unavoidably incurs an area overhead.

Inter-chiplet networks adapt **network-on-chip (NoC)** principles for multi-chiplet systems, incorporating tailored approaches for topology, routing,¹ and deadlock handling. Network topology defines the connections among network interfaces and routers. Figure 5 illustrates canonical hand-crafted topologies. Early work finds that topologies like Double Butterfly offer advantages in router count, network diameter, and bisection bandwidth over Concentrated Mesh and Mesh on silicon interposers [79]. Building on this, researchers design ButterDonut that fuses Double Butterfly and Folded Torus to enhance bandwidth [88]. Additionally, the Kite topology family is proposed to account for the impact of link length on clock frequency [10]. For large-scale systems with hundreds of chiplets, the HexaMesh topology is introduced to significantly reduce network diameter and enhance bisection bandwidth over Mesh [71]. A routing algorithm determines a packet's path from source to destination, significantly influencing latency and link utilization. Flexible routing can direct packets through either chiplet or interposer networks to improve performance, balance loads based on factors like hop count and congestion, and increase reliability [79, 155, 227]. Various strategies are employed to prevent network deadlocks. These include turn-restriction methods for both application-agnostic and application-specific systems [216, 217], using virtual networks for resource isolation [191], and other techniques like flow control [134] or deadlock recovery [212].

2.2 Advantages and Limitations

Chiplet technology can sustain the advancement of chip integration density. Unlike monolithic architectures constrained by reticle limits, multi-chiplet systems achieve horizontal expansion through 2D and enhanced-2D packaging. Waferscale chips, such as Dojo [192], Big Chip [60], and

¹Based on the context, the term routing can refer to the logical paths for network packets or the physical paths for wires.

waferscale network switch [21], demonstrate the extreme potential of this approach. Furthermore, vertical expansion via 3D packaging not only reduces footprint but also significantly shortens inter-chiplet interconnects, improving performance and power efficiency [127].

Chiplet technology offers several avenues for reducing design and manufacturing costs. Modular chiplets can be reused across multiple products, effectively amortizing high **non-recurring engineering (NRE)** costs [39]. Additionally, their smaller die sizes improve yields and enhance wafer utilization. Components that do not scale well with advanced process nodes (e.g., I/O circuits and SRAM) can be fabricated on separate chiplets using cost-effective mature processes. As a result, the **recurring engineering (RE)** costs can also be optimized [143, 213].

Chiplet technology can accelerate chip development and time-to-market. By decomposing designs into reusable chiplets, designers can develop multiple chips for diverse niche markets with fewer tape-outs and less design effort. Additionally, designers can integrate existing chiplets to rapidly construct target systems. Furthermore, the benefits of chiplet technology extend to the manufacturing stage. Fabricating individual chiplets on different process nodes allows for the simultaneous use of multiple foundry lines, thereby significantly cutting production time [147]. Therefore, the chiplet-based agile design and integration can be seen as an extension of existing agile chip development flows, which mainly focus on reusing pre-silicon RTL creation and verification [215], into the realms of physical design and post-silicon processes.

Chiplet technology can enhance supply chain security and robustness. First, it allows manufacturing only a part of the design at an untrusted high-end foundry and the remaining part at a trusted low-end foundry [74, 207]. This split manufacturing approach mitigates risks like reverse engineering and hardware Trojans, which arise from entrusting entire designs to a single foundry. Second, the modularity of multi-chiplet systems offers the flexibility to substitute components with alternative chiplets, thereby improving resilience against supply and demand volatility [89, 147].

However, chiplet technology inevitably introduces overheads, known as **chiplet tax** [110]. First, the cross-die communication requires D2D interfaces and TSVs, which add to the chiplet area. For example, D2D interfaces occupy nearly 10% of the die area in the AMD Zeppelin processor [9]. Second, in 2D and enhanced-2D packaging, D2D communication involves complex operations such as clock domain crossing and error detection, increasing data transfer latency and energy consumption, often by an order of magnitude compared to on-chip communication [174]. Third, multi-chiplet systems often adopt more complex packaging technologies, leading to higher costs and potentially lower packaging yields. Furthermore, dense vertical stacking in 3D packaging poses severe thermal management challenges, necessitating more expensive cooling solutions [34]. Consequently, chiplet tax may outweigh savings, particularly for products with small die areas or those using mature process nodes [110]. This highlights the urgent need for chiplet design automation tools to help designers navigate these tradeoffs and maximize overall benefits.

2.3 Evaluation Model

Chiplet design flows use evaluation models to assess intermediate results across various metrics. The speed and accuracy of these models directly impact the quality of system parameters derived from **design space exploration (DSE)**. Below are the evaluation models for common metrics.

2.3.1 Performance Model. The performance of multi-chiplet systems critically depends on the inter-chiplet network. While detailed simulators like Booksim [80] and Garnet [3] are accurate, their serial nature makes them impractical for large-scale systems. To accelerate simulation, Feng et al. [46] introduce a packet-centric parallel simulation technique that assigns packets to different threads to effectively leverage multi-core processors. Its high computational complexity, however,

Table 2. Summary of Performance Models

Cited Work	Key Method	Strengths	Limitations
[3, 80]	Serial simulation	High accuracy	Impractical for large-scale systems
[46]	Packet-centric parallel simulation	Parallel execution	High computational complexity
[72]	Analytical model	High speed	Low accuracy (neglects flow control)
[109]	Data-driven approach	High accuracy and high speed	Network-level only (lacks holistic view)
[226]	Unified co-simulation framework	Full-system evaluation	High integration complexity
[63]	Expanded co-simulation platform	Broader support (e.g., Verilog, FPGA)	High integration complexity
[118, 208]	Application-specific model (DNN)	Rapid end-to-end evaluation	Domain-specific (limited applicability)

remains a bottleneck for rapid DSE. As a faster alternative, the analytical model proposed by Iff et al. [72] considers only D2D link delay, neglecting flow control effects and thus providing only a rough estimation of network latency and throughput. To improve accuracy without sacrificing speed, Li et al. [109] introduce a data-driven approach, representing network topology and traffic as an attributed graph and using a **graph neural network (GNN)** for precise performance prediction. Network-level analysis alone is insufficient for a holistic performance view. A unified framework that co-simulates compute, memory, and network components is required for comprehensive system-level evaluation. To this end, Zhi et al. [226] develop an inter-simulator communication and synchronization protocol, enabling the integration of various chiplet simulators for parallel full-system simulation. A subsequent platform, Switchboard [63], adopts a similar principle but expands its support beyond software models to include Verilog simulation and FPGA emulation. For multi-chiplet systems targeting specific applications, performance evaluation can be simplified. For example, HISIM [208] and HEX-SIM [118] relate **deep neural network (DNN)** layer execution latency to chiplet parameters and layer parallelization strategies. By combining this with analytical inter-chiplet network models, rapid end-to-end performance evaluation for DNN accelerator systems is enabled. To provide a clear overview, we summarize and compare the key performance modeling paradigms in Table 2.

2.3.2 Cost Model. The cost of a multi-chiplet system comprises NRE and RE costs. NRE costs are one-time expenses for design, IP licensing, and masks. In contrast, RE costs are per-chip expenses for die fabrication, packaging, and testing. Consequently, the amortized NRE cost is the dominant factor for low-volume production, whereas the RE cost dictates the expense at high volumes.

The NRE cost is modeled by summing the contributions from IP, chiplet, and system levels. To estimate the NRE costs for IPs and chiplets, Khazraee et al. [92] develop a model relating parameters such as transistor count, process node, and application requirements to components like labor and IP expenses. For the system-level NRE cost, Feng et al. [45] express it as the sum of a fixed component and a variable component that scales linearly with package area.

The RE cost mainly encompasses chiplet fabrication and packaging expenses. The chiplet fabrication cost is given by $C_{die} = (C_{wafer}/N_{die} + C_{test})/Y_{die}$. Here, C_{wafer} represents the wafer cost (tied to the process node), N_{die} is the number of diced chiplets per wafer (a function of chiplet area),

Table 3. Summary of Cost Models

Cost Type	Cited Work	Focus Level	Key Parameters	Strengths	Limitations
NRE	[92]	IP/Chiplet	Transistor count, process node, application requirements	Detailed correlation of low-level parameters	Limited to IP/chiplet level
	[45]	System	Fixed component, package area	High-level system abstraction	Simplified linear relationship assumption
RE	[34, 111]	System	Wafer cost, die area, test cost, yield, substrate cost, bonding cost	Accounts for D2D/TSV area overhead	Omits complex assembly/test flows
	[5, 56, 186]	Complex packaging	Assembly sequence, test flow, cooling solution	Covers complex assembly/test flows	High modeling complexity

C_{test} is the testing cost, and Y_{die} denotes the manufacturing yield (influenced by both chiplet area and process node). Notably, the chiplet area accounts for the overhead of D2D interfaces and TSVs [34, 111]. Both wafer utilization and yield decrease as chiplet area increases, leading to a higher cost per unit area, particularly at advanced nodes [45]. The packaging stage, where multiple tested chiplets are bonded to a carrier, further contributes to the RE cost. For a 2D system with n chiplets, the packaging cost is $(C_{org} + \sum_{i=1}^n (C_{die_i} + C_{bond}))/Y_{bond}^n$. Here, C_{org} is the organic substrate cost, C_{die_i} is the cost of each chiplet, C_{bond} is the per-chiplet bonding cost, and Y_{bond} is the bonding yield for an individual chiplet. For more complex packaging technologies, advanced RE cost models consider the assembly sequence, test flow, and cost of cooling components [5, 56, 186].

Table 3 summarizes these key cost models, comparing their focus, primary parameters, and respective tradeoffs.

2.3.3 Physical Model. Thermal models describe the temperature distribution in a multi-chiplet system during operation. Stow et al. [186] develop a temperature modeling approach for 2.5D and 3D systems, considering factors such as ambient temperature, thermal resistance, and chiplet power density. However, this method only provides system-level estimates and lacks detailed temperatures for individual components. To overcome this limitation, Park et al. [160] employ **finite element method (FEM)** simulations for precise local temperature assessments. However, its high computational overhead significantly reduces the efficiency of DSE. Pfromm et al. [164] further propose various simplified FEM models to estimate temperatures with varying accuracy and speed across different design stages. Additionally, Chen et al. [19] introduce a GNN model trained on extensive thermal data, enabling rapid thermal evaluations and improving DSE efficiency.

Reliability models assess the probability of component failure in multi-chiplet systems, often employing multi-physics simulations to capture the interplay of thermal, mechanical, and electrical effects. A primary reliability concern is warpage arising from coefficient of thermal expansion (CTE) mismatches between chiplets and packaging materials. The analytical model proposed by Irwin et al. [75] indicates that warpage is influenced by CTE differences, temperature changes, and system scales. For more granular analysis, Hisada et al. [64] use FEM to investigate the impact of assembly sequence and interposer material on warpage. Another critical reliability issue is bump stress. High stress can deform bumps, potentially compromising power delivery and I/O functions [231]. An

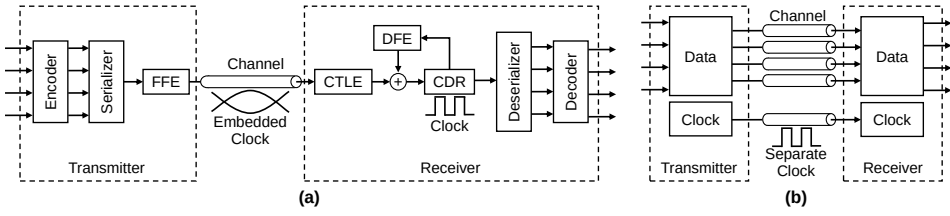


Fig. 6. Microarchitecture of (a) serial interface and (b) parallel interface.

Table 4. Summary of Physical Models

Modeling Metric	Cited Work	Specific Concern	Key Method	Strengths	Limitations
Thermal	[186]	System-level temperature	System-level model	Models system-level temperature distribution	Lacks component-level detail
	[160]	Local temperature	FEM	Models local temperature with high precision	High computational overhead hinders DSE
	[164]	Multi-stage temperature estimation	FEM	Simplified FEM models	Offers lower accuracy than full-fidelity FEM
	[19]	Rapid thermal evaluation	GNN	Rapid thermal evaluation	Requires extensive training data
Reliability	[75]	Warpage	Analytical model	Models warpage analytically	Lacks detailed, localized analysis
	[64]	Warpage	FEM	Detailed warpage analysis	High computational cost
	[85]	Bump stress	Semi-analytical method	Models the variation of bump stress	Less accurate than direct FEM simulation
	[152]	TSV failure	Electromigration lifetime model	Models the electromigration lifetime of TSVs	Specific to the electromigration failure mode
	[84, 85]	TSV failure	FEM	Models thermo-mechanical TSV failure	High computational cost

existing model [85] shows that bump stress is influenced by the position and size of bumps. In 3D systems, TSV failure presents an additional challenge. Pak et al. [152] model the electromigration lifetime of TSVs based on current, temperature, and TSV-induced stress. Furthermore, models by Jung et al. [84, 85] utilize FEM to correlate the probability of thermo-mechanical TSV failure with both individual TSV dimensions and the overall array layout. To provide a systematic overview of these approaches, Table 4 summarizes the key physical models for thermal and reliability analysis.

2.3.4 Circuit Model. SerDes-based serial D2D interfaces achieve a drastic reduction in I/O pin count by increasing data transfer rates to over 100 Gb/s per lane [51]. This high performance, however, introduces substantial signal integrity and noise challenges. Modern SerDes address them through complex mechanisms like clock embedding and signal equalization. As depicted in Figure 6(a), a typical serial interface comprises a transmitter, a channel, and a receiver. The

transmitter integrates an encoder, a serializer, and a **feed-forward equalizer (FFE)** to mitigate signal distortion over long distances. In addition to the decoder and deserializer, the receiver employs a **decision feedback equalizer (DFE)** and a **continuous-time linear equalizer (CTLE)** to further correct the signal. A **clock and data recovery (CDR)** circuit is necessary for extracting the clock embedded in the data stream. Mainstream SerDes modeling relies on statistical analysis and time-domain simulation [95]. Statistical analysis assumes linear time-invariance and uses system transfer functions for quick signal quality assessment in low **bit-error-rate (BER)** conditions. Time-domain simulation, while more computationally expensive, offers higher accuracy by modeling non-linear equalization and CDR effects via transient analysis. In practice, these methods are often combined to accommodate the varying properties of different components. To further enhance evaluation efficiency, Nguyen et al. [146] leverage data sequences from circuit simulators to train recurrent neural networks (RNNs) for direct eye diagram prediction. Subsequently, Dong et al. [33] introduce a framework designed for signal-power integrity co-analysis in high-speed serial links. It integrates ML-based transmitter models and impulse response-based models for the channel and receiver.

In contrast to high-speed serial counterparts, parallel D2D interfaces simplify architecture by employing a significantly larger number of interconnects that operate at lower per-lane data rates. As depicted in Figure 6(b), a typical parallel interface uses single-ended signaling with a dedicated clock channel, a design that eliminates the need for complex equalization and CDR circuits. Modeling these interfaces is correspondingly straightforward. Existing methods [29, 153] represent transmitters and receivers as cascaded inverter chains, model transmission lines with distributed **resistor-capacitor (RC)** networks, and treat components like **electrostatic discharge (ESD)** circuits and microbumps as fixed capacitive elements. This approach enables the rapid assessment of key performance metrics, including interface bandwidth and data transmission energy. The **Universal Chiplet Interconnect Express (UCIe)** has emerged as the predominant industry standard formalizing this parallel D2D approach. At the PHY level, UCIe provides a standardized interface capable of carrying complex protocols for interoperability, whereas direct signaling relies on simpler, often proprietary, parallel I/O for maximum efficiency over short distances.

For multi-chiplet systems, regardless of whether they employ serial or parallel interfaces, ESD protection is a critical component of the circuit model. Graening et al. [55] demonstrate that the required ESD protection for the proliferation of I/O interfaces in chiplet systems creates critical area and power overheads. This tradeoff establishes a practical lower limit on cost-effective chiplet size.

For 3D systems, the complex D2D PHYs described above can be dramatically simplified or eliminated. This PHY-less paradigm is realized in two main forms. First, technologies like hybrid bonding provide ultra-high-density X-Y surface connections with minimal parasitics, enabling true PHY-less interfaces driven directly by standard logic gates. Second, vertical communication relies on TSVs (the Z-axis path), which, when terminated with microbumps, have slightly higher parasitics and typically require simple dedicated buffers rather than a full PHY [78]. Modeling these 3D interconnects is crucial. Katti et al. [90] characterize individual TSVs based on physical dimensions and material characteristics. Liu et al. [121] further model TSV-to-TSV coupling, conducting full-chip signal integrity analysis to assess its impact on noise and timing.

To synthesize the preceding discussion on inter-chiplet communication, Table 5 offers a comparative summary of these three primary interface paradigms. It systematically contrasts the serial, parallel, and PHY-less approaches across key attributes, clarifying the critical design tradeoffs involved in selecting an appropriate interface technology.

2.3.5 Other Models. Recent research in chiplet technology has extended beyond traditional metrics to evaluate security, environmental impact, and time-to-market.

Table 5. Summary of Circuit Models

Interface Type	Key Characteristics	Core Components	Key Method	Complexity
Serial	High data rate per lane, low pin count, complex signal integrity challenges	Tx/Rx with complex equalization and CDR	Statistical analysis, time-domain simulation, or ML models	High
Parallel	Low data rate per lane, high pin count, simpler architecture	Simple drivers and passive RC channels	Simple circuit models for drivers and channels	Low
PHY-less	Vertical communication, highest density, minimal parasitics	TSVs, digital buffers, and fine-pitch bonds	Physical characterization and full-chip coupling analysis	Low at the logic level, but high at the physical modeling and integration level

In split manufacturing, security evaluation models are essential for guiding hardware design partitioning. For instance, Imeson et al. [74] and Fyrbiak et al. [50] introduce security metrics based on subgraph isomorphism and graph edit distance, respectively. However, these approaches are primarily applicable to 3D systems. To address this limitation, Safari et al. [169] propose a metric based on graph similarity to support 2D and enhanced-2D systems.

Another emerging focus is the life-cycle carbon footprint of multi-chiplet systems. Zhao et al. [223] present a model that quantifies emissions during manufacturing, considering factors such as chiplet area, process nodes, and packaging technologies. The model also incorporates operational emissions, which depend on the system's power consumption and lifetime. Sudarshan et al. [188] further integrate the carbon footprint of the EDA tools used during the design phase.

The cost model in Section 2.3.2 concentrates on monetary expenses, but time-to-market is also vital for product competitiveness. Ning et al. [147] propose a model that connects the time needed for chip fabrication, manufacturing, and packaging with factors like chiplet transistor count, manufacturing technology node, and foundry capacity. This model helps in selecting parameters to mitigate the impact of foundry capacity fluctuations on time-to-market.

3 Top-down Design Flow

3.1 Overview

3.1.1 Problem Formulation. As illustrated in Figure 7, the top-down design flow ² aims at generating a multi-chiplet system configuration that maximizes the user-defined optimization objective. The primary input is a hardware design, which can be manually crafted or automatically generated using techniques such as hardware templates [107] or **Large Language Models (LLMs)** [14], and is modeled as follows:

Definition 1. A hardware description graph is a directed graph $\mathcal{G}_H(\mathcal{V}_H, \mathcal{E}_H)$, where each vertex $v_i \in \mathcal{V}_H$ represents a hardware component (e.g., an IP block or a gate), and each directed edge $e_{i,j} \in \mathcal{E}_H$ denotes a connection from component v_i to v_j .

²This flow focuses on hardware exploration of multi-chiplet systems, assuming that software scheduling is handled separately at the upstream design-time stage [22] or the downstream run-time stage [128].

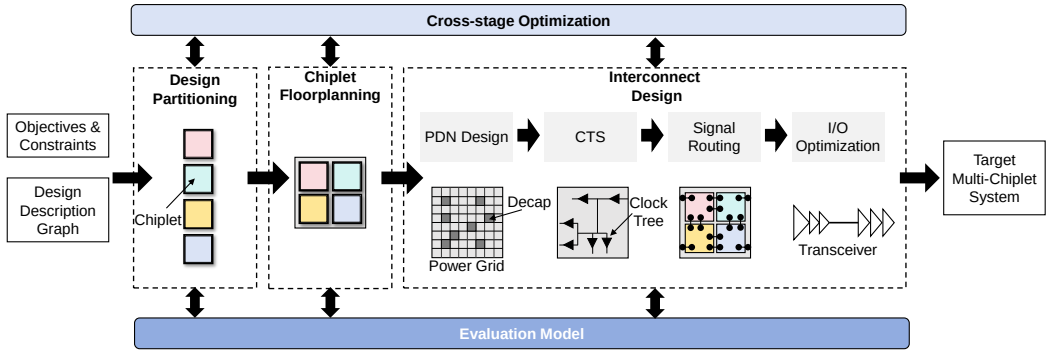


Fig. 7. Overview of a typical top-down chiplet design flow.

The design flow comprises multiple stages and allows for cross-stage optimization. Initially, the input hardware design is partitioned into multiple parts, a process formally defined as

Definition 2. Given the hardware description graph $\mathcal{G}_H(\mathcal{V}_H, \mathcal{E}_H)$, design partitioning divides the vertex set $\mathcal{V}_H$ into a set of k subsets $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$, such that $\bigcup_{i=1}^k S_i = \mathcal{V}_H$.³ Each subset S_i corresponds to the hardware components assigned to a chiplet C_i .

The partitioning can also be extended to co-optimize multiple input designs to identify reusable chiplets across a product line [111, 170]. Subsequently, the floorplanning stage determines the spatial arrangement for partitioned chiplets.

Definition 3. Given the partition $\mathcal{S}$, chiplet floorplanning generates a set $\mathcal{F}$ of tuples that defines both the intra-chiplet and inter-chiplet layouts. Each tuple specifies the coordinate (2D or 3D), orientation (e.g., vertical or horizontal), and dimension (i.e., width and height) for either an internal component $v_j \in S_i$ or an entire chiplet C_i . The final layout must satisfy physical constraints, including non-overlapping placement, maximum aspect ratio, and minimum inter-die spacing.

Following floorplanning, the physical interconnects are synthesized to establish connections for power, clock, and signals. This critical phase is typically divided into four sub-stages: **power delivery network (PDN)** design, **clock tree synthesis (CTS)**, signal routing, and I/O optimization.

Definition 4. Given the chiplet floorplan $\mathcal{F}$ and its associated power demands, PDN design determines the parameters of power stripes and TSVs, such as dimensions and locations. This process must satisfy constraints such as IR drop, electromigration, and thermal limits.

Definition 5. To achieve timing closure for the given chiplet floorplan $\mathcal{F}$ and sink locations, CTS creates a clock distribution network by first generating an abstract clock tree topology, which is a graph $\mathcal{G}_C(\mathcal{V}_C, \mathcal{E}_C)$ where $\mathcal{V}_C$ represents the internal nodes and $\mathcal{E}_C$ denotes the logical connections. Then, clock embedding/routing determines the precise physical locations of the internal nodes and the paths for the connecting wires.

With the partition $\mathcal{S}$, inter-chiplet signal nets are edges that cross chiplet boundaries, i.e., the set $\mathcal{E}_{inter} = \{e_{i,j} \in \mathcal{E}_H \mid v_i \in S_u, v_j \in S_k, u \neq k\}$. These nets connect D2D interfaces on different chiplets using microbumps. Additionally, the system communicates with external devices through nets $\mathcal{E}_{ext}$, which connect off-package interfaces to C4 bumps on the package substrate. Signal routing determines the physical paths for both types of nets and is formulated as:

³This constraint can be relaxed to $\mathcal{V}_H \subseteq \bigcup_{i=1}^k S_i$ to improve chiplet reusability in IP-level partitioning [170].

Definition 6. Given the chiplet floorplan $\mathcal{F}$ and all signal nets $\mathcal{E}_{inter} \cup \mathcal{E}_{ext}$, signal routing finds their exact paths on **redistribution layers (RDLs)**. For 3D systems, this also includes planning the locations of TSVs. The final routing must satisfy all design rules, such as minimum metal spacing, maximum layer count, permissible routing angles, and non-crossing wire requirements.

Once the signal routes are established, the parameters of D2D interfaces can be tuned to further enhance link performance and signal integrity. This leads to the I/O optimization stage:

Definition 7. For each D2D interface, I/O optimization seeks to find an optimal parameter set describing aspects of the transceivers (e.g., driver size, equalizer settings) or the channels (e.g., wire dimensions, spacing). The optimization must adhere to physical and electrical design constraints while meeting signal integrity targets.

3.1.2 Design Space. The preceding definitions delineate a vast design space explored during the top-down flow, which can be conceptualized across three layers:

- Logical Layer: At the highest level of abstraction, this layer focuses on system architecture. Key decisions involve partitioning the hardware graph into a set of chiplets, which directly impacts manufacturing cost, reusability, and the communication bandwidth required between chiplets (Definition 2).
- Physical Layer: This layer translates the logical partition into a tangible layout by determining the spatial arrangement of chiplets and their internal components, and by planning interconnects for power, clock, and signals. Optimizing this layer is crucial for managing timing, wirelength, and thermal dissipation (Definitions 3, 4, 5, and 6).
- Circuit Layer: At the most granular level, this layer addresses the electrical characteristics of the D2D links. It involves the detailed parameter tuning of I/O transceivers and channels to ensure signal integrity, meet performance targets, and minimize power consumption (Definition 7).

3.2 Design Partitioning

Whereas traditional SoC partitioning decomposes a design on a single process node primarily to accelerate synthesis, design partitioning for chiplets seeks the optimal system-level tradeoff between cost and performance by enabling the integration of chiplets from diverse and specialized process nodes. This process assigns hardware components to chiplets, critically affecting chiplet cost and inter-chiplet communication bandwidth. Figure 8 illustrates two main types based on the granularity of hardware description graph: coarse-grained IP-level (vertices are IP blocks, edges are inter-IP communications) and fine-grained gate-level (vertices are logic gates, edges are nets). While finer granularity allows for more flexible resource distribution, it demands higher interconnect density and lower latency [127]. For 2D and enhanced-2D packaging, the large parasitics of interconnects might reduce system clock frequency when separating logic gates into different chiplets [86]. In contrast, 3D packaging can effectively support gate-level schemes by using low-RC vertical interconnects. To address this complex, multi-objective challenge, a wide range of partitioning algorithms have been developed, which are summarized and contrasted in Table 6.

Figure 8 showcases key IP-level partitioning solutions, primarily aimed at costing optimization. Uniform partitioning (❶) [45, 187] divides homogeneous multi-core designs into identical chiplets, determining the optimal number via enumeration. However, this approach is unsuitable for systems with diverse IPs. For heterogeneous designs, the IP-based method (❷) [157] assigns each IP to a separate chiplet. This often leads to too many small chiplets, which increases packaging costs and inter-chiplet communication. Focusing on RE cost optimization, the balanced partitioning scheme

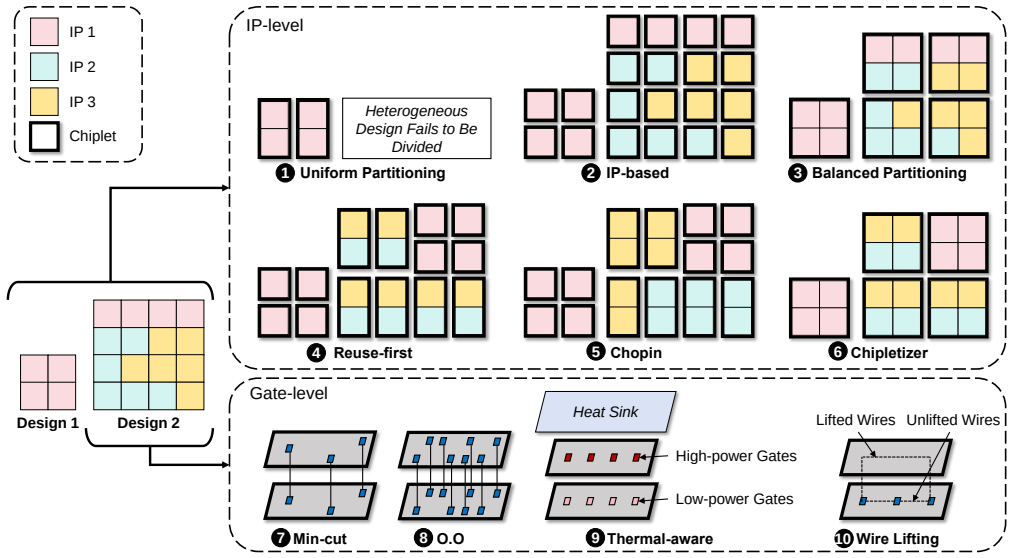


Fig. 8. Comparison of various partitioning strategies. Here, only the hardware components are shown, and the communication among IPs is omitted. The coarse-grained IP-level partitioning can be applied to multiple designs, whereas the fine-grained gate-level partitioning normally focuses on single design.

(3) [86] utilizes min-cut algorithm to enhance die yields and reduce inter-chiplet traffic, albeit compromising chiplet reusability and raising NRE costs. Conversely, the reuse-first method (4) [111] prioritizes reducing amortized NRE costs by identifying minimal reusable chiplet templates via **integer linear programming (ILP)**. However, this might inflate RE costs and communication overhead. Chopin (5) [39] aims at co-optimizing NRE and RE costs by partitioning same-type IPs into chiplets based on a total cost model. Its effectiveness is limited as it cannot combine different IP types, which hinders communication optimization. It also depends on inaccurate estimates of reusable chiplet volume, potentially misleading NRE cost optimization. More advanced methods, including Partition-Merge [170] and Chipletizer (6) [111], co-partition multiple hardware designs within a product line to explore reusable chiplets. They flexibly aggregate frequently communicating IPs onto the same chiplet and consider the partitioning's impact on subsequent stages like placement, ultimately optimizing overall system cost, performance, and power.

Figure 8 also illustrates various gate-level partitioning methods developed for 3D systems. Unlike 2D and enhanced-2D packaging that allow flexible horizontal chiplet placement, vertically stacked 3D systems face more stringent area constraints. In response, many approaches continue to use balanced partitioning (7) to reduce cross-die interconnects and maintain area balance [18, 81, 101]. However, these methods do not always yield optimal partitions in terms of total wirelength and timing, because they fail to fully leverage the high density and short vertical interconnects [162]. O.O (8) [196] adapts the min-cut algorithm by incorporating a multi-objective gain formulation that additionally considers the **half-perimeter wirelength (HPWL)** of cuts. Beyond wirelength and area, other evaluation metrics are also considered. Due to severe thermal challenges, the thermal-aware strategy (9) [8] is proposed to strategically position high-power units near heat sinks. For security, wiring lifting (10) [74] focuses on partitioning wires of the netlist. It isolates sensitive wires into trusted chiplets to enhance resistance to attacks. To manage the increased computational complexity associated with fine-grained gate-level partitioning, researchers are increasingly leveraging techniques like deep learning [130].

Table 6. Summary of Design Partitioning Strategies

Partition Granularity	Cited Work	Optimization Objective	Strengths	Limitations
IP-level	[45, 187]	Cost	Simplified method	Unsuitability for heterogeneous systems
	[157]	IP independence	High die yield	High costs and high inter-chiplet communication overhead
	[86]	Inter-chiplet traffic	High die yield	Low reusability high NRE costs
	[111]	NRE costs	High reusability	Increased RE costs
	[39]	NRE costs	Low cost	Inability to integrate heterogeneous IPs
	[111]	Cost, latency and power (Weighted sum)	Supporting heterogeneous IP integration	Methodological complexity
Gate-level	[18, 81, 101]	Cross-die interconnects	Balanced area	Insufficient optimization in wirelength and timing
	[196]	Wirelength, area and interconnects (Weighted sum)	Low wirelength	Increased vertical interconnect count
	[8]	Thermal management capability	Scalability to large-scale systems	Lack of wirelength constraint
	[74]	Security	Strong attack resistance	High computational overhead

Both IP-level and gate-level partitioning have evolved from naive, connectivity-based methods to sophisticated, physically-aware optimizations. At the IP-level, this improvement is now primarily channeled into enhancing cost-effectiveness, with a focus on optimizing for yield and reusability. Meanwhile, gate-level partitioning has become more concerned with maximizing performance, tackling latency and thermal challenges. For complex heterogeneous systems, a hierarchical co-simulation approach that integrates both IP-level and gate-level partitioning represents a promising direction. This hybrid strategy allows IP models to be used for global architecture exploration to maintain simulation efficiency, while selectively applying gate-level partitioning to performance-critical regions to capture precise timing and thermal behaviors without incurring prohibitive runtime.

3.3 Chiplet Floorplanning

Chiplet floorplanning fundamentally differs from conventional monolithic IC floorplanning by introducing a hierarchical challenge. While monolithic design involves placing macros and standard cells

Table 7. Summary of Floorplanning Optimization Methods

Package Type	Cited Work	Data Structure	Optimization Objective	Key Method
2D/Enhanced-2D	[178]	Scalar spacing	Cost and thermal (Weighted sum)	SA-based
	[65]	B*-tree	Wirelength, area, and cost (Weighted sum)	SA-based
	[111]	B*-tree	Wirelength	SA-based
	[168]	B*-tree	Negative slack, cost, wirelength, and area (Constraint method)	SA-based and RL-based
	[73]	Order and orientation	Wirelength, area, and cost (Weighted sum)	GA-based and SA-based
	[26]	Sequence Pair	Wirelength	Branch-and-bound
	[123]	Sequence Pair	Wirelength	Enumeration-based
	[151]	CSP	Wirelength	Branch-and-bound
	[66]	TCG	Warpage	SA-based
	[20]	Coordinate	Wirelength, area, warpage, and cost (Weighted sum)	Mathematical programming
	[202]	Coordinate	Wirelength and thermal (Pareto optimization)	Gradient descent
	[231]	Coordinate	Wirelength, cost, and warpage (Weighted sum)	Mathematical programming
	[133]	Coordinate	Wirelength and thermal (Constraint method)	SA-based
	[103]	BT-tree	Wirelength and area (Weighted sum)	SA-based
	[171]	Coordinate	Wirelength and area (Weighted sum)	SA-based
	[140]	Sequence Pair	Area and thermal (Weighted sum)	Bayesian optimization
	[37]	Coordinate	Wirelength and thermal (Weighted sum)	RL-based
	[194]	Coordinate	Power, performance and cost (Constraint method)	LLM-based
3D	[28]	TCG	Wirelength, area, TSV quantity, and thermal (Weighted sum)	SA-based
	[115]	Coordinate	Wirelength	Gradient descent
	[49]	Coordinate	Wirelength	Gradient descent

on a single die to optimize metrics like wirelength and congestion, chiplet floorplanning operates on two distinct levels: (1) inter-chiplet floorplanning, which arranges multiple, often heterogeneous dies on a package substrate or interposer, and (2) intra-chiplet floorplanning of internal components like standard cells and I/O buffers⁴ within each individual die. The 3D packaging adds a vertical dimension, extending the floorplanning process beyond planar placement. These two levels are conceptually similar, as both involve two key aspects: data structures for placement description and optimization algorithms for finding the optimal arrangement. Table 7 summarizes existing chiplet floorplanning methods.

⁴In this article, the I/O buffer refers to a D2D interface or an off-package interface, for consistency with other literature.

Different data structures offer tradeoffs between flexibility and solution space size. For planar chiplet placement, Shukla et al. [178] propose a simplified method that utilizes a fixed chiplet aspect ratio and controls chiplet spacing with a scalar parameter. This approach offers high search efficiency but is restricted to regular grid arrangements of homogeneous chiplets. To describe more irregular placements, B*-trees, which encode the relative positions of chiplets, are commonly employed [65, 111, 168]. However, B*-trees inherently support only compacted layouts where no chiplet can be moved further left or down. Similarly, another proposed data structure describes chiplet placement order and orientation [73]. While amenable to **genetic algorithms (GAs)**, this approach is also confined to compacted arrangements. Such limitations restrict the direct insertion of gaps for thermal optimization, often necessitating post-processing steps. To support a broader range of placements, more general representations like Sequence Pair [26, 123], Constraint-Satisfaction Problem [151], and **Transitive Closure Graphs (TCGs)** [66] are adopted. Alternatively, some methods directly store chiplet coordinates without relying on relative positioning. These include approaches using continuous coordinates [20, 202, 231] and those discretizing the placement plane into grids [133]. While offering maximum flexibility, they can lead to an explosively large solution space, thereby requiring customized optimization algorithms to accelerate the search process. Addressing adjacency and overlapping constraints of silicon bridge-based packaging, Lee et al. [103] propose the BT-tree, combining the TCG with B*-tree to manage complex layout constraints while significantly reducing computational complexity. These 2D data structures can be extended to support 3D cell placement. For example, Cong et al. [28] use an array of TCGs to represent 2D placement on each layer, while recent works often use 3D coordinates for cell positioning [115].

Optimization algorithms seek the optimal placement within the solution space defined by the data structure, targeting objectives like minimizing wirelength, optimizing chiplet aspect ratios, and managing thermal dissipation. They can be broadly categorized into search-based and analytical ones. Search-based methods explore the solution space with various strategies. Liu et al. [123] propose an enumeration-based method that effectively handles small-scale systems. However, the computational complexity of exhaustive enumeration becomes prohibitive as the number of chiplets increases. To mitigate this, branch-and-bound approaches that prune unpromising configurations based on their wirelength are introduced [26, 151]. Random search techniques such as **simulated annealing (SA)** [178] and Bayesian optimization [140] can adapt to various data structures. However, these methods typically start each search from scratch, which can limit their efficiency. To overcome this, RL-based methods [37, 168] have been used to learn from historical optimization experience. Conversely, analytical approaches leverage the explicit structural details of the placement to directly optimize chiplet or standard cell coordinates. For example, many works construct differentiable approximate objective functions for metrics like wirelength or temperature, enabling efficient coordinate optimization via gradient descent [49, 202]. An LLM agent has also been applied to assist placement in MAHL [194], automating the iterative refinement of parameters by interpreting error logs from EDA tools. Other strategies formulate the placement task as a mathematical programming problem, which is then solved using commercial solvers [20, 231].

In summary, chiplet floorplanning is evolving beyond traditional discrete representations and purely heuristic methods, as such approaches struggle to handle the increasingly detailed requirements of modern designs. This shift is driven by the fact that the scope of optimization has expanded far beyond simple wirelength minimization. A successful floorplan is now judged by a suite of multi-physics criteria, where die-to-die interconnect performance, power delivery, and thermal integrity are treated as critical objectives. In this context, analytical methods leveraging gradient descent currently demonstrate the superior scalability and quality for such high-dimensional design

Table 8. Summary of PDN design methods

Package Type	Cited Work	Optimized Parameters	Optimization Task	Key Method
2D/ Enhanced-2D	[68]	Number, size, and coordinates of microbumps, width and placement of power stripes	Satisfying IR drop constraint	Iteration-based
	[221]	Decap layout	Minimizing impedance and decap count (Weighted sum)	GA-based
	[159]	Decap layout	Minimizing area subject to target impedance	ML-based
	[158]	Decap layout	Minimizing area subject to target impedance	ML-based
	[138]	Power island layout	Optimizing PDN impedance	ML-based
3D	[228]	Width and pitch of power stripes, and track utilization	Minimizing wire resistance subject to IR drop constraint	ML-based
	[203]	Number, size, and coordinates of TSVs	Minimizing average IR drop subject to area constraint	MILP-based
	[195]	PDN metal track width and TSV diameter	Minimizing PDN/TSV area subject to electrothermal constraints	Iteration-based
	[204]	Number and size of TSVs, and number of deactivated TSVs	Maximizing EM lifetime subject to IR drop constraint	Heuristic-based

spaces compared to stochastic heuristics. In the future, significant opportunities lie in extending these continuous frameworks to support complex physical constraints, potentially augmented by reinforcement learning to jointly optimize discrete architectural decisions.

3.4 Interconnect Design

The primary distinction between chiplet and monolithic IC design lies in the hierarchical challenge of the interconnects. By distributing a system across multiple chiplets, designers face a two-level problem: connecting components internally on each chiplet and connecting the chiplets to one another. This duality reshapes every aspect of interconnect design, from the PDN and CTS that must span the entire assembly, to the now-separate tasks of intra-chiplet and inter-chiplet routing, and especially the I/O optimization, which now focuses on the critical D2D interfaces that enable communication between chiplets.

3.4.1 PDN Design. The primary objective of PDN design is to minimize area, subject to rigorous constraints on IR drop, electromigration, and thermal performance. Table 8 provides a comparative overview of associated optimization approaches.

To mitigate IR drop, Huang et al. [68] propose a design flow to co-optimize the placement and dimensions of both microbumps and power stripes. The flow utilizes k-means clustering for initial placement, followed by an iterative loop of nodal analysis and analytical model-based adjustments to satisfy the specified voltage constraint. For many designs, the physical dimensions of PDNs are strictly constrained by the **process design kit (PDK)**, which leaves limited room for modification. Recognizing this, other efforts have concentrated on optimizing decoupling capacitor (decap) placement to reduce the PDN impedance. Zhang et al. [221] introduce a thermal and power integrity co-optimization framework employing an improved adaptive GA. Their two-stage approach first performs thermally-aware placement, then uses the transmission matrix method

to characterize PDN impedance and optimizes the on-chip decap layout to suppress impedance while minimizing capacitor overhead to prevent overdesign. **Reinforcement learning (RL)** has emerged as a powerful tool for PDN impedance optimization in 2.5D systems. While early RL-based approaches from Park et al. [158, 159] have successfully addressed decap layout optimization, their scope is typically limited to this single component, neglecting the power and ground routing. More recent work by Miao et al. [138] broadens this scope, employing a Dueling Double Deep Q Network to co-optimize the layout of both the PDN and its power routes. They innovatively incorporate squareness, i.e., the geometric shape of power islands, into their reward function, which guides the RL agent toward more optimal PDN shapes.

PDN design becomes more challenging in 3D systems. These systems require more metal layers for vertical power delivery but accommodate fewer C4 bumps compared to 2D systems, which complicates the optimization process [13]. To address these 3D-specific issues, researchers have developed specialized automated flows. Zhu et al. [228] propose Hetero-3D, an ML-based flow that integrates an area-unbalanced 3D floorplanner with PDN and signal routing optimization. Their PDN strategy creates a discrete solution space of parameters such as stripe width and pitch, uses Gaussian Process Regression to predict routing cost and IR drop, and employs the L-BFGS algorithm to identify optimal settings that minimize resistance while adhering to voltage constraints. As TSVs are an integral part of the 3D PDN, their optimal planning is crucial for ensuring a reliable power supply design. For TSV optimization, Wang et al. [203] employ a **mixed-integer linear programming (MILP)** algorithm to optimize the location, size, and count of TSVs, thereby minimizing PDN IR drop while adhering to constraints on the total TSV area. Addressing various reliability constraints like electromigration, Todri et al. [195] propose a tier-based method for 3D PDNs that co-optimizes both metal track widths and TSV diameters. This co-optimization directly manages the tradeoff between the power delivery benefits of larger TSVs and their consumption of silicon area and routing resources, while satisfying temperature and voltage drop limits. Wang et al. [204] focus on the transient recovery effect in electromigration stress evolution. By using more small-diameter TSVs and periodically deactivating some TSVs, they achieve an optimized balance among the current density, electromigration recovery effect, and TSV diameter.

PDN design for chiplets fundamentally differs from traditional approaches by introducing new physical components and multi-level optimization goals. In 2.5D architectures, this involves co-designing on-chip elements with interface structures like microbumps and decoupling capacitors. In 3D architectures, the challenge evolves to include vertical power pathways, making the strategic planning of TSVs a primary design consideration alongside the conventional metal grid. Among the presented strategies, ML-based approaches, particularly reinforcement learning, demonstrate superior capability in handling these high-dimensional search spaces. However, ensuring reliability often incurs high simulation costs. Future research could thus focus on hybrid co-simulation frameworks that integrate rapid ML inference with precise physics-based solvers, capturing the advantages of both methods to enhance the efficiency of PDN synthesis in complex chiplet designs.

3.4.2 CTS. The primary objective is to deliver the clock signal to all sequential elements within chiplets while meeting timing constraints and minimizing skew, delay, and power consumption. These representative CTS methods are summarized in Table 9.

For 2D and enhanced-2D systems, CTS generally involves two main stages: abstract tree topology generation and embedding/routing it with buffer insertion. The topology generation stage adapts to the large scale and hierarchical nature of these systems. For instance, Figliolia et al. [48] propose the Conical-Fishbone clock tree, whose radially symmetric topology ensures identical impedance at all points on a given clock ring to achieve very low skew. For high-performance computing chiplets with even stricter timing requirements, Lin et al. [117] introduce a three-level clock

Table 9. Summary of CTS Methods

Package Type	Cited Work	CTS Problem	Optimization Objective	Key Method
2D/ Enhanced-2D	[48]	Low-skew clock tree topology generation	Clock skew and power (Via architecture design)	Conical-Fishbone clock tree architecture
	[117]	Advanced-node clock tree topology generation	Clock skew	Three-level clock tree architecture (H-tree + mesh + local CTS)
	[70]	Inter-chiplet delay tuning	Clock skew	Adjustable drivers
	[24]	Inter- and intra-chiplet latency equalization	Clock skew	DLLs with online calibration
	[142]	Inter-chiplet clock distribution	Clock signal integrity and power (Via architecture design)	Hierarchical clock architecture (reference clock + on-chiplet PLLs)
3D	[97]	Bounded-skew clock tree topology generation and embedding	Wirelength (Under skew constraint)	Bounded-skew clock tree with deferred merge embedding
	[139]	Thermal-aware clock tree topology generation	Skew variation and wirelength (Weighted sum)	Buffered clock tree with thermal optimization
	[189]	Timing convergence degradation in multi-chiplet stacking	Timing closure and energy efficiency (Via design methodology)	3D clock forwarding and hierarchical CTS rebalancing
	[98]	Cost-effective clock tree topology generation, layer assignment, and tree embedding	TSV count, wirelength, and buffer count (Weighted sum)	NN-3D for topology generation, and DLE-3D/DME-3D for layer assignment and embedding
	[163]	Low-skew clock tree topology generation	Wirelength, TSV count, and skew (Weighted sum)	Ternary tree topology generation
	[167]	Hierarchical clock tree topology generation with obstacle avoidance	Clock power, phase delay, and skew (Hierarchical co-optimization)	Hierarchical clock architecture (global GH-tree + local binary trees)
	[129]	Low-power gated clock tree topology generation and tree embedding	Power	K-means for topology generation and SA-based clock tree routing
	[149]	Thermal-aware clock tree topology generation, layer assignment and tree embedding	Clock skew	3D-NNM for topology generation and thermal-aware 3D-DME for tree routing

architecture. This combines a global H-tree for driving pre-placed mesh drivers, a robust mesh structure to minimize on-chip variation, and local clock trees to serve sinks. The clock tree routing stage faces the significant challenge of synchronizing clock signals across different chiplets. One strategy involves actively compensating for a high-frequency clock distributed directly across the interposer. For example, Huang et al. [70] use a self-tuning process with adjustable drivers to correct for inter-chiplet delay variations. A more comprehensive method from Cheng et al. [24] utilizes **delay-locked loops (DLLs)** to dynamically compensate for both inter- and intra-chiplet latencies via automatic online calibration. However, the strategy of distributing a high-frequency clock directly can be power-intensive and susceptible to signal integrity issues. To mitigate this, an alternative approach proposed by Murali et al. [142] routes a low-frequency reference clock across the interposer, which is then scaled up locally on each chiplet by **phase-locked loops (PLLs)**, effectively reducing high-frequency signal degradation and crosstalk.

3D CTS introduces the additional dimension of assigning tree nodes to different vertical layers [224]. For the topology generation stage, top-down methods like 3D-MMM extend the 2D **method of means and medians (MMM)** algorithm [76] to recursively partition sinks. Building on the MMM-based abstract tree topology, Kim et al. [97] focus on skew-bounded clock tree embedding, while Minz et al. [139] address buffered clock tree synthesis under thermal variations. Sumbul et al. [189] employ the 3D clock-forwarding methodology for dual-die stacks that creates non-divergent and predictable cross-die timing paths to overcome synchronization challenges caused by process variations. However, accurately predicting subtree characteristics in a top-down manner is challenging. Consequently, bottom-up approaches like **nearest neighbor selection (NN-3D)** [98] have been developed, which recursively merge pairs of sinks or subtrees based on a merging cost. Expanding on this, Pavlidis et al. [163] utilize cost-driven ternary trees to further reduce clock skew. Ran et al. [167] propose a hierarchical method combining a global **generalized H-tree (GH-tree)** with local binary trees that incorporate obstacle avoidance capabilities. For the clock tree routing stage, Kim et al. [98] introduce **deferred layer embedding (DLE-3D)** and **deferred merge embedding (DME-3D)** to respectively minimize TSV cost and total wirelength. Furthermore, specialized 3D CTS flows have been developed to address specific design goals. For low-power design, Lu et al. [129] propose a two-phase method using k-means clustering to group nodes by activity and proximity, followed by an SA-based embedding phase with simultaneous shutdown gate insertion. To manage thermally-induced skew, Oh et al. [149] develop a flow that starts with the nearest-neighbor selection with median cost (3D-NNM) topology to cluster sinks with similar power consumption, followed by an even distribution of TSVs for layer assignment and a final thermal-aware 3D-DME routing.

Unlike monolithic design, CTS for chiplets must address the core problem of synchronizing multiple, physically separate clock domains. In 2.5D systems, this introduces the new challenge of managing inter-chiplet skew, which is addressed using global distribution strategies involving DLLs or PLLs. In 3D systems, the problem becomes three-dimensional, making the optimal placement of TSVs and layer assignment as critical as the tree topology itself. The fundamental shift is from a planar routing exercise to a complex, multi-domain problem that must account for new physical structures (like TSVs) and system-level synchronization across the entire package. To address these challenges, hierarchical architectures that combine global invariant structures (e.g., meshes or H-trees) with local flexible trees currently offer the best balance between skew minimization and power efficiency. Furthermore, future CTS approaches could leverage machine learning-assisted dynamic calibration, co-optimizing active compensation circuits (like DLLs/PLLs) with physical routing.

3.4.3 Signal Routing. A signal net is a conceptual representation of a set of terminals that must be electrically connected. The primary objective of the signal routing stage is to plan the

Table 10. Summary of Signal Routing Methods

Package Type	Cited Work	Routing Problem	Key Method
2D/ Enhanced-2D	[43]	Free-assignment single-layer flip-chip RDL routing	Network-flow
	[41]	Free-assignment multi-layer flip-chip RDL routing	Network-flow
	[42]	Pre-assignment single-layer flip-chip RDL routing	ILP
	[105]	Pre-assignment single-layer flip-chip RDL routing	WLCS and MPSC
	[44]	Unified-assignment single-layer flip-chip RDL routing	DT for network-flow graph construction
	[116]	Unified-assignment multi-layer multi-chip RDL routing	Concentric-circle model for layer assignment and network-flow
	[25]	Pre-assignment multi-layer multi-chip RDL routing with variable wire width	LCP
	[210]	Unified-assignment via-based multi-layer multi-chip RDL routing	Weighted MPSC-based layer assignment and network-flow
	[27]	Pre-assignment any-angle multi-layer multi-chip RDL routing	Access point adjustment and tile routing in detail routing
	[230]	Free-assignment multi-layer multi-chip RDL routing with timing constraint	Grid-based maze routing, network-flow, and iterative refinement
3D	[124]	Regular TSV assignment	Shortest path, bipartite matching, MCMF, and ILP
	[93]	Irregular TSV assignment	Analytical-based
	[61]	Irregular TSV assignment	Multilevel refinement
	[59]	Irregular TSV assignment	MADRL

specific physical paths of metal wires for each net. In the context of package-level routing, this process includes connecting intra-chiplet I/O buffers to microbumps, connecting microbumps within different chiplets, and connecting microbumps to C4 bumps. The routing problem is broadly classified into three types based on how terminals are assigned to nets: **free-assignment (FA)**, **pre-assignment (PA)**, and **unified-assignment (UA)** [104]. In the FA problem, the router may flexibly select the optimal terminal for each signal net from a given set of candidate pads. This flexibility enables global optimization, often resulting in improved performance and reduced total wirelength. Conversely, the PA problem mandates that the terminals for each net are predetermined before routing begins. This approach offers designers greater control and predictability during high-level planning, but it presents a more significant algorithmic challenge due to operating under fixed constraints. The UA problem is a hybrid model that includes both freely-assigned and pre-assigned nets. This is a common scenario in multi-chiplet systems. For instance, critical inter-chiplet connections might be pre-assigned while connections to the external environment are left flexible. By accommodating both conditions, UA routing provides a framework to balance performance optimization with specific design constraints. A summary of common algorithms developed for these routing problems is presented in Table 10.

Fang et al. [43] propose a routing algorithm for FA problems, which consists of global routing and detailed routing phases. During global routing, a network-flow graph is constructed based on the spatial distribution of pins and terminals, followed by a **minimum-cost maximum-flow (MCMF)** algorithm to assign terminals and plan paths for all signal nets. In the detailed routing phase, methods such as cross-point assignment, net ordering, and track assignment are employed to

complete the routing. However, this algorithm assumes that pins are always located at the edges of chiplets. They thus extend this network-flow-based algorithm to handle arbitrary pin distributions, achieving smaller package sizes and reduced wirelength while improving signal and power integrity [41]. The PA routing problem corresponds to the multi-commodity flow problem and cannot be directly solved by network-flow-based algorithms [104]. Fang et al. [42] propose an ILP-based global routing algorithm with pruning techniques to address the PA routing problem. Lee et al. [105] propose an efficient global routing algorithm by computing the **weighted longest common subsequence (WLCS)** and the **maximum planar subset of chords (MPSC)**. This approach, based on dynamic programming, significantly improves computational efficiency over ILP-based methods. Fang et al. [44] propose a solution for the UA routing problem. It employs **Delaunay triangulation (DT)** and Voronoi diagrams from computational geometry to construct the network flow graph and estimate routing congestion, simultaneously completing the routing for both PA and FA signal nets. Lin et al. [116] address the UA problem for multi-chip, multi-layer RDL routing. They introduce a concentric-circle model to manage layer assignment for pre-assigned nets and then integrate this geometrical information into a network-flow model to handle free-assigned nets, thereby minimizing both total wirelength and the number of layers used. Traditional routing algorithms partition the routing space into uniform grids, which cannot support non-uniform wire widths and spacing. To address this, Chiang et al. [25] introduce a gridless routing framework that transforms the routing problem into a **linear complementarity problem (LCP)**. They solve this problem using the **modulus-based matrix splitting iteration method (MMSIM)** and a bipartite matching algorithm, thereby enabling unified solutions for both gridded and gridless routing. Wen et al. [210] address the UA routing problem for multilayer packages with flexible vias and irregular pad structures. Their algorithm performs a weighted MPSC-based layer assignment for concurrent inter-chiplet routing and utilizes a network-flow model for subsequent chip-to-board connections. Chung et al. [27] present the first any-angle routing algorithm for multiple RDLs. Their approach includes a detailed routing stage featuring a multi-net access point adjustment method based on dynamic programming, followed by an efficient tile routing algorithm to finalize the connections. Zhuang et al. [230] propose a timing-driven co-optimization routing algorithm for chiplet-package co-design. It iteratively optimizes the package and chiplet routing to maximize routability, minimize via count and wirelength, and ensure the routing result conforms to the timing constraints.

Compared to 2D/enhanced-2D packaging, where signal nets are routed within the package plane, 3D packaging requires additional planning for TSV assignment in multi-layer stacked structures. The pattern of TSV placement can be categorized into regular and irregular placement. In regular placement, target TSVs are selected from pre-arranged fixed-pitch arrays to form interconnects across layers. Liu et al. [124] propose an integrated framework combining shortest-path, bipartite matching, maximum-flow minimum-cut, and ILP to reduce interconnect costs. Although regular placement reduces computational complexity, its strict grid pattern limits the potential for further optimization. This motivates research into irregular TSV placement, where TSVs can be freely distributed across layers. Kim et al. [93] propose a framework that first inserts TSVs into partitioned netlists and then optimizes their positions using a force-directed algorithm. For large-scale designs, this method can face excessive runtime. To accelerate the process, Hao et al. [61] adopt a hierarchical approach, which first partitions the TSV placement region into multi-level grids and then refines the locations of TSVs from their initial positions. Guan et al. [59] employ a **multiagent deep reinforcement learning (MADRL)** approach, modeling the assignment problem as a Markov decision process and outperforming heuristic methods.

The hierarchical structure of signal routing for chiplets defines its distinction from monolithic design, separating the problem into on-die routing and, more critically, inter-chiplet routing on

the package. This new package-level domain often requires solving complex unified-assignment problems to accommodate both fixed and flexible connections. In 3D architectures, the challenge becomes three-dimensional, introducing TSVs as a new and critical routing resource that must be managed. The fundamental shift is from optimizing paths within a single plane to coordinating routes across multiple distinct physical components, including the chiplets, the RDL, and the vertical TSV links that tie them together. Among the reviewed strategies, hybrid frameworks that integrate global planning (e.g., network flow) with detailed gridless or any-angle routing demonstrate the most effective balance between routability and wirelength minimization. However, the increasing density of interconnects poses scalability challenges for these traditional solvers. Future research should therefore explore chiplet-package co-optimization techniques, potentially augmented by deep learning, to simultaneously address the routing demands of both the internal dies and the external package layers within a unified simulation environment.

3.4.4 I/O Optimization.

3.4.4.1 Optimizing Serial Interfaces. Section 2.1 introduces various D2D interfaces designed to balance interconnect density, performance, power efficiency, and communication reach across diverse applications and packaging technologies. Table 11 summarizes research optimizing circuit parameters for these D2D interface types in specific multi-chiplet systems. The following sections detail these optimization approaches, categorized by interface architecture.

Given the intricate nature of SerDes-based D2D interfaces, a significant body of research focuses on optimizing the parameters of individual components. First, transmitter optimization involves tuning FFE parameters to enhance signal integrity at the source. For instance, Chang et al. [15] employ a GA-based method to optimize the number of FFE taps and their coefficients, aiming at maximizing eye height. However, this approach overlooks metrics like eye width, which is crucial for jitter tolerance. To address this limitation and further accelerate the optimization process, Ling et al. [119] propose a machine learning-driven framework. It utilizes a Random Forest algorithm to train a decision model that adjusts FFE taps and associated coefficients, explicitly optimizing eye height or width. Second, receiver optimization seeks optimal parameters for equalization mechanisms, enabling accurate data recovery by mitigating channel-induced distortions, attenuation, and noise. Dehlaghi et al. [32] propose a GA-based method to adjust the parameters of programmable capacitors and resistors in the **continuous-time linear equalizer (CTLE)**. To support more complex receivers with high-dimensional parameters, Zhu et al. [229] construct a dataset from simulation and apply **Principal Component Analysis (PCA)**. This technique reduces the dimensionality of the control parameters for the CTLE, **decision feedback equalizer (DFE)**, and FFE, effectively improving the convergence speed of the online optimization algorithm for **bit error rate (BER)** minimization. Third, channel optimization seeks to improve data transmission quality by adjusting inter-chiplet interconnect parameters. Kim et al. [94] use a GA-based method to tune physical dimensions of differential signal lines and material properties of the dielectric layer, minimizing channel loss and crosstalk. To shorten the time for circuit simulation, they construct a neural network-based surrogate model for rapid prediction of transmission line properties such as resistance and inductance. Targeting a broader design space, Manukovsky et al. [135] employ an ML-based feature range analysis method to maximize **Channel Operating Margin (COM)**. It identifies key influencing factors from numerous channel-related parameters, such as transmission line dimensions, dielectric constant, and dielectric layer spacing and height. The method subsequently derives their optimal ranges.

A growing trend in serial interface design is the move toward holistic optimization. For instance, Radi et al. [165] jointly optimize receiver and channel elements, including FFE, DFE, **transimpedance amplifier (TIA)**, T-coil, and transmission lines. The optimization process incorporates

Table 11. Summary of I/O Optimization Methods

Interface Type	Cited Work	Optimized Interface Component	Optimized Parameters	Optimization Objective	Key Method
Serial	[15]	Transmitter	FFE tap numbers and coefficients	Eye height	GA-based
	[119]	Transmitter	FFE coefficients	Eye height or width	ML-based
	[32]	Receiver	Programmable resistances and capacitances of CTLE	Eye opening	GA-based
	[229]	Receiver	Control parameters for CTLE, DFE, and FFE	BER	ML-based
	[94]	Channel	Physical dimensions of differential signal lines and material properties of the dielectric layer	Channel loss and crosstalk (Weighted sum)	GA-based
	[135]	Channel	Transmission line dimensions, dielectric constant, dielectric layer spacing and height	COM	ML-based
	[165]	Channel and receiver	Transmission line width, T-coil geometry, TIA design parameters, FFE and DFE tap weights	SNR	GA-based
	[16]	Channel, transmitter, and receiver	Transmission line width and spacing, transceiver types	Energy per bit	Enumeration-based
	[131]	Channel, transmitter, and receiver	Transmission line dimensions and control parameters for CTLE, DFE, and finite-impulse response filter	Eye width or height	ML-based
Parallel	[156]	Channel	Transmission line width and pitch, layer thickness	Bandwidth density	Iteration-based
	[153]	Channel, transmitter, and receiver	Microbump dimensions, wire and link dimensions, number of metal routing layers	Latency and energy per bit	Enumeration-based
	[211]	Channel	Microbump pitch	Signal/Power/Thermal integrity, and wire length (Weighted sum)	ML-based
	[29]	Channel	Insertion locations of chiplet registers	Wirelength	MILP-based
PHY-less	[106]	Channel	Number and locations of buffers, location of TSVs	Path delay	Heuristic-based
	[23]	Channel	TSV driving buffer size	Total buffer area	ILP-based and heuristic-based
	[122]	Channel	Size of the KOZ	Coupling noise and path delay (Weighted sum)	Enumeration-based
	[209]	Channel	Microbump thickness and height, TSV diameter	TSV resistance and von Mises stress (Pareto optimization)	GA-based

key metrics such as jitter and noise into a **signal-to-noise ratio (SNR)** metric to enhance overall signal integrity. Focusing on the entire D2D interfaces, Chaudhary et al. [16] enumerate all combinations of transceiver types, transmission line widths and spacings to optimize energy per bit. However, this exhaustive approach exhibits limited scalability. For exploring larger design spaces, Ma et al.

[131] introduce an active subspace optimization method leveraging support vector regression. This data-driven technique establishes an implicit correlation linking high-dimensional transmitter and receiver parameters to performance metrics. It transforms complex, high-dimensional, nonlinear constrained problems into more manageable low-dimensional, linear constrained ones, enabling inverse problem-solving from target constraints to design parameters.

3.4.4.2 Optimizing Parallel Interfaces. Parallel D2D interfaces feature simpler structures and fewer design parameters compared to serial ones, resulting in lower optimization complexity. Guided by an analytical bitrate model, Pantano et al. [156] optimize bandwidth density by sweeping candidate values of channel parameters including line width, line pitch, and layer thicknesses. Following this, Pal et al. [153] broaden the design space by considering additional channel characteristics like microbump pitch and by adjusting transceiver settings such as sizing cascaded inverters. Woo et al. [211] first employ neural network-based models to predict the impacts of microbump pitch on signal, power, and thermal integrity. Leveraging these predictions, they then use Bayesian optimization to determine the optimal microbump pitch according to target specifications. The effectiveness of this approach is subsequently validated with actual designs. The above discussions are limited to single-hop data transmission in passive interposers. To support long-distance transmission in such systems, Coskun et al. [29] propose using flip-flops within intermediate chiplets to enable multi-cycle pipelined communication. Their proposed MILP-based algorithm optimizes the insertion locations of these chiplet registers during routing, aiming at minimizing wirelength under the constraint of maximum register number for a route. In active silicon interposers, buffers/repeaters can be inserted to extend the signal transmission distance within a single clock cycle, which is similar to the method used for on-chip buffer insertion [6].

3.4.4.3 Optimizing PHY-less Interfaces. Advanced 3D integration leveraging fine-pitch interconnects eliminates the need for sophisticated interface circuits, a trend exemplified by the UCIE-3D standard [31]. It adopts a PHY-less design approach, leveraging hybrid bonding to radically simplify the PHY into a minimal electrical interface that allows direct logic connection and removes the need for a complex D2D adapter. By doing so, it achieves high bandwidth density and power efficiency, as Cyclic Redundancy Check and retry mechanisms become unnecessary over the vertical channel. However, even with this simplified interface architecture, the underlying buffers and TSVs of the channels still require significant refinements. Since TSVs create non-uniform resistance and capacitance along the net paths, conventional on-chip buffer insertion methods [6] become inapplicable. To adapt to 3D systems, Lee et al. [106] first propose an analytical delay estimation model for 3D interconnects. Then they decide the optimal number and locations of inserted buffers with a fast heuristic algorithm. Nevertheless, this work does not consider the significant impact of crosstalk between densely packed TSVs, focusing primarily on RC delay reduction. Addressing this limitation, Chen et al. [23] develop a TSV delay model that incorporates crosstalk effects from neighboring aggressor TSVs, considering various buffer sizes. Based on this model, they introduce both an ILP-based method for achieving near-optimal buffer sizing and a more computationally efficient crosstalk-aware heuristic algorithm. Their objective is to select optimal buffer sizes that satisfy timing constraints while minimizing the total buffer area. Several studies optimize TSV dimensions to enhance performance and reliability. Liu et al. [122] study the impact of the **keep-out-zone (KOZ)** size around TSVs on coupling noise and path delay. They propose an enumeration-based KOZ sizing method and a placement refinement technique using a weighted force function to mitigate TSV-to-TSV coupling issues and reduce path delay. To enhance thermo-mechanical reliability, Wang et al. [209] employ response surface approximation and simulation data. They express maximum von Mises stress and TSV copper resistance as functions of design parameters, i.e., TSV diameter and microbump dimension. A multi-objective GA algorithm then uses these expressions to

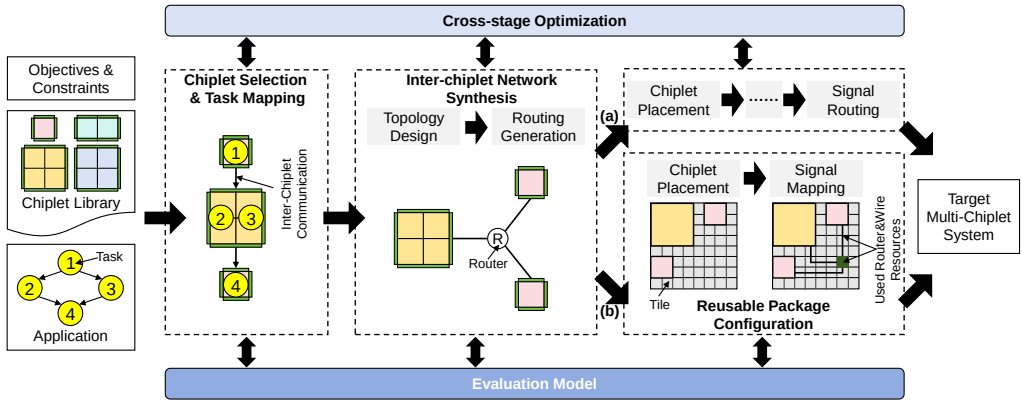


Fig. 9. The bottom-up chiplet design flows based on (a) custom packages and (b) reusable packages.

identify Pareto-optimal parameters that balance the tradeoff between reducing stress and improving electrical conductivity.

The key difference in I/O optimization for chiplets versus monolithic chips is the shift from standard chip-to-board I/O to a diverse set of specialized D2D interfaces. This requires optimizing the entire communication channel as a system, including co-designing transceivers and physical traces in serial links, tuning the dimensions of microbumps in parallel links, and managing the physical properties of TSVs in 3D integration. Ultimately, the design challenge moves beyond simple signal driving to a multi-physics optimization problem focused on maximizing bandwidth and power efficiency within the unique constraints of the package environment. Currently, holistic strategies that simultaneously tune transmitter, channel, and receiver parameters outperform component-wise methods, particularly when accelerated by machine learning to navigate high-dimensional design spaces. However, the reliance on computationally expensive electromagnetic simulations remains a bottleneck. Future work should focus on end-to-end co-simulation frameworks that integrate multi-physics solvers with active learning algorithms, enabling efficient synthesis of robust high-bandwidth interfaces across varying package technologies.

4 Bottom-up Design Flow

This section focuses on the bottom-up chiplet design methodology, which uses existing validated chiplets to rapidly develop multi-chiplet systems that meet application requirements. We start by outlining the overall design flow and formulating the associated problems. Subsequently, we survey the relevant design automation technologies developed for this methodology.

4.1 Overview

4.1.1 Problem Formulation. The bottom-up design flow shown in Figure 9 aims at maximizing the user-defined optimization objective. It takes an application specification and a library of available chiplets as its primary inputs, which are formally defined as follows:

Definition 8. A chiplet library [144] is a set $\hat{\mathcal{C}}$, where each element $\hat{C}_i$ represents a unique type of chiplet with defined attributes (e.g., functionality, area, and cost). An application is described by a directed acyclic graph $\mathcal{G}_T(\mathcal{V}_T, \mathcal{E}_T)$. Each node in $\mathcal{V}_T$ is a task with specific resource demands, and each edge in $\mathcal{E}_T$ denotes the data dependency between two tasks.

The design flow begins with selecting a suitable set of chiplets from the library and mapping the application's tasks onto these selected chiplets.

Definition 9. Chiplet selection determines the multiset of chiplets to be used, represented as $\mathcal{S}_{sel} = \{(\hat{C}_i, n_i) | \hat{C}_i \in \hat{\mathcal{C}}, n_i \in \mathbb{N}\}$, where n_i is the number of instances of chiplet type $\hat{C}_i$. The result of task mapping is a function $\mathcal{M} : \mathcal{V}_T \rightarrow \mathcal{C}'$, where $\mathcal{C}'$ is the set of chiplet instances derived from $\mathcal{S}_{sel}$. The resulting mapping must satisfy all resource and data dependency constraints.

Once tasks are allocated to specific chiplets, an inter-chiplet network is synthesized to handle the resulting communication traffic. The topology and routing are common network parameters considered in design automation, which are formulated as:

Definition 10. The inter-chiplet network topology is a directed graph $\mathcal{G}_I(\mathcal{V}_I, \mathcal{E}_I)$, where $\mathcal{V}_I$ is the set of network nodes (e.g., routers and network interfaces) and $\mathcal{E}_I$ is the set of links connecting them. Routing defines the paths that packets traverse from source to destination nodes, described by a function $\mathcal{R} : \mathcal{V}_I \times \mathcal{V}_I \rightarrow \mathcal{P}_I$, where $\mathcal{P}_I$ is the set of all possible paths (i.e., link sequences).

This logical network is then realized physically through package-level implementation, involving the placement of chiplets and routing of inter-chiplet interconnects. This process can follow two pathways. The first, shown in Figure 9(a), uses a custom package design flow as described in Section 3. While this method offers maximum flexibility, designing and manufacturing custom packages, especially those with active interposers, incur substantial NRE costs and a long development cycle. This requirement fundamentally hinders the agile integration that is a key motivation for the bottom-up paradigm. To address these challenges, the concept of package reuse has gained significant attention, as shown in Figure 9(b). Reusable packages offer configurable interconnects organized as tiles, enabling the synthesized inter-chiplet network to be implemented through assembly-time configuration. This eliminates the design and manufacturing stages for the package, which drastically reduces costs and time-to-market. It is worth noting that while top-down flows can also leverage reusable packages, their primary bottlenecks are often the chiplet design and tape-out phases, which significantly limit the overall benefit gained from package reuse.

4.1.2 Design Space. The design space for a bottom-up flow leveraging reusable packages can be conceptualized across three distinct layers:

- Logical Layer: At the highest level of abstraction, this layer is concerned with system-level architectural decisions. Key parameters include the selection of chiplet types and quantities from the library, the mapping of application tasks onto the selected chiplets, and the synthesis of the inter-chiplet network's topology and routing (Definitions 9, 10).
- Physical Layer: This layer is responsible for translating the logical architecture into a physical implementation. It involves determining the placement of the selected chiplets onto the reusable package and mapping their signal nets by configuring the package's built-in communication fabric.
- Circuit Layer: This layer is highly constrained because the bottom-up design flow utilizes existing chiplets, whose I/O circuits are typically fixed and offer limited optimization space.

4.2 Chiplet Selection and Task Mapping

For application-specific systems, selecting chiplets and mapping tasks are tightly coupled processes that aim at optimizing for performance, area, and cost while adhering to hardware constraints.

The diversity of the chiplet library directly influences the complexity of the selection process. Table 12 provides a detailed overview of the state-of-the-art methods for chiplet selection. When the library consists of homogeneous chiplets, the selection process simplifies to determining the optimal number of chiplets. For instance, Krishnan et al. [100] calculate the required number of **in-memory computing (IMC)** chiplets per DNN layer based on computational resource constraints, and sum these to find the total system requirement. When the library contains heterogeneous chiplets, both

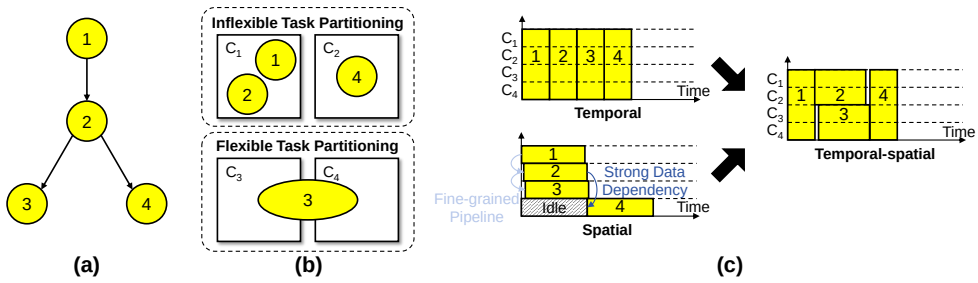


Fig. 10. Illustration of task mapping strategies. (a) An example task graph. (b) Inflexible and flexible task partitioning on a four-chiplet system. (c) Space-time diagrams for temporal, spatial, and temporal-spatial execution models. The fine-grained pipeline allows for nearly complete execution overlap between predecessor and successor tasks, whereas the strong data dependency requires the predecessor task to completely finish before its successor can begin.

Table 12. Summary of Chiplet Selection Methods

Chiplet Library Type	Cited Work	Key Method	Strengths	Limitations
Homogeneous	[100]	Analytical	Simplified method	Poor scalability
Heterogeneous	[99]	Exhaustive search	High hardware utilization	Poor scalability
	[108]	ILP-based	Accurate multi-objective matching	High complexity
	[154]	ILP-based	Low cost and high flexibility	High simulation overhead

the types and quantities of chiplets must be determined. In their later work, Krishnan et al. [99] consider heterogeneous IMC systems with big and little chiplets, aiming at enhancing hardware utilization. However, their approach of enumerating all possible combinations does not scale well for larger libraries. For a library comprising various computation and memory chiplets (e.g., CPUs and DRAMs), Li et al. [108] propose an ILP-based method that effectively accelerates the solution process by leveraging commercial solvers. Moving to a more intricate problem, Pal et al. [154] consider the design of multiple systems for multiple applications. Specifically, they address selecting chiplets from a library containing CPUs and cache chiplets to construct k multi-chiplet systems for n applications. When $k = 1$, this problem simplifies to designing a general-purpose system that performs optimally on average across all applications. When $k = n$, it involves customizing a system for each application. Their research highlights that allowing multiple applications to share the same chiplet combination can significantly reduce total costs without substantial performance loss, thus achieving a balance between customization and generality.

Task mapping strategies are influenced by the flexibility of task partitioning. As depicted in Figure 10(b), inflexible task partitioning treats each task as an indivisible unit confined to a single chiplet [99, 100, 108, 205]. This constraint leads to suboptimal performance, as multiple chiplets cannot be used to accelerate bottleneck tasks, especially when tasks have divergent computational or memory demands. Mapping strategies also depend on the task execution model. As shown in Figure 10(c) and Table 13, task execution methods are categorized as temporal, spatial, and temporal-spatial. Temporal execution involves sequentially processing tasks using the chiplet resources of the entire system. For instance, Tan et al. [193] partition a single DNN layer across

Table 13. Summary of Task Mapping Strategies

Execution Type	Cited Work	Key Method	Strengths	Limitations
Temporal	[193]	Cross-chiplet layer partitioning	Enhanced performance	Limited hardware utilization
	[218]	Cross-layer data reuse consideration		
Spatial	[108]	Constraint-aware ILP formulation	Memory efficiency and performance enhancement	Unsuitability for resource-constrained systems
Temporal-spatial	[11]	Segment-grained dynamic programming formulation	Enhanced flexibility and efficiency	Single application focus
	[205]	ILP formulation	High resource utilization and efficiency	Limited to homogeneous multi-core systems
	[35]	Redundant cross-chiplet communication elimination	Heterogeneous support	Single application focus
	[220]	Model block partitioning and exhaustive search	Parallel scheduling of multiple DNN models on multi-chiplet systems	Homogeneous system constraint
	[148]	Concurrent execution of multiple DNN models	Enhanced performance	Risk of local optima
	[77]	Communication-aware adaptive approach	High flexibility with communication load awareness	Weak adaptability to dynamic scenarios

all chiplets, optimizing the partitioning pattern and loop transformation while considering inter-chiplet communication. Zhang et al. [218] further improve performance by considering data reuse among sequentially executed layers. The drawback of this approach is the inability to saturate hardware resources when executing fine-grained tasks on large-scale systems, leading to low utilization. Spatial execution, conversely, maps tasks to different chiplets for concurrent execution. In certain circumstances, spatial execution allows fine-grained inter-task pipelining, reducing the on-chip memory footprint and improving performance. For example, Li et al. [108] formulate an ILP problem with constraints on computational, memory, and D2D bandwidth resources to determine the chiplet assignment for each task. However, such one-to-one mapping is ill-suited for resource-constrained systems and leads to idle resources when tasks exhibit strong data dependencies. To address these limitations, hybrid temporal-spatial mapping methods have gained popularity due to their enhanced flexibility and efficiency. Cai et al. [11] use dynamic programming to partition DNN models into segments that execute sequentially, while layers within each segment execute spatially through pipelining. Extending beyond the DNN domain, Wang et al. [205] introduce an ILP-based method for arbitrary homogeneous many-core systems. Meanwhile, Du et al. [35] focus on heterogeneous systems and eliminate redundant inter-chiplet communication during optimization. While these methods optimize for single-application scenarios, emerging applications like autonomous driving require multi-chiplet systems to concurrently run multiple DNN models with diverse resource demands and latency constraints. Addressing this challenge, Zhang et al. [220] first partition DNN models into blocks and then employ an exhaustive search to determine

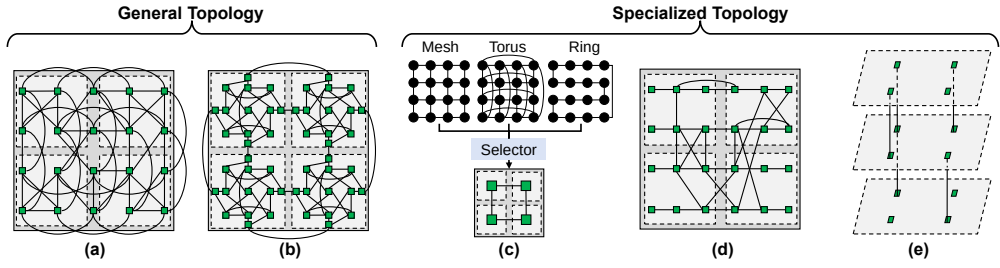


Fig. 11. Synthesized inter-chiplet network topologies, where dotted boxes represent chiplets and green boxes denote routers. Application-agnostic: (a) NetSmith and (b) fault-tolerant. Application-specific: (c) selection-based, (d) directly-generated 2.5D, and (e) directly-generated 3D.

their execution order and chiplet allocation. This research is confined to homogeneous systems, whereas Odema et al. [148] further boost performance by scheduling DNN layers onto chiplets with varying dataflow architectures. Similarly, targeting the complex phases of LLMs, Jaiswal et al. [77] introduce a scalable, communication-aware mapping algorithm, which minimizes the product of communication volume and Manhattan distance by adaptively employing either a branch-and-bound or a sequential placement strategy, demonstrating a highly flexible, workload-aware approach.

A comprehensive chiplet library enables designers to explore diverse hardware combinations, and sufficiently fine-grained application partitioning is a critical prerequisite for effective task mapping. Advancing both these foundational aspects is essential for achieving better performance and cost efficiency. In addition, addressing design agility will likely require alternatives to exact solvers (e.g., ILP) with limited scalability, such as fast learning-based frameworks that can adapt to different application workloads.

4.3 Inter-chiplet Network Synthesis

In bottom-up design flows, the inter-chiplet network must be synthesized around a set of physical constraints imposed by selected chiplets, which have predetermined I/O configurations. Within these stringent limitations, the topology and routing must then be carefully co-designed to satisfy the communication traffic requirements of the mapped tasks.

4.3.1 Topology Generation. According to communication patterns, inter-chiplet network topology generation can be classified into application-agnostic and application-specific methods, as shown in Table 14.

Application-agnostic methods aim at creating general-purpose network topologies that handle unknown and potentially random traffic effectively [57]. While numerous expert-designed topologies exist (see Section 2.1), this manual approach fails to thoroughly explore the large design space and struggles to balance various performance and overhead metrics. To address these challenges, Green et al. [57] propose NetSmith, which is an ILP-based topology generation framework that holistically optimizes metrics including average hop count and saturation throughput. The resulting topology, shown in Figure 11(a), achieves significantly higher throughput with a lower average hop count compared to previous expert-designed networks. To generate fault-tolerant topologies, Wang et al. [206] utilize an RL-based optimization that begins with a fully connected graph and iteratively removes links. The process maximizes network robustness, which is defined as the minimum percentage of faulty or attacked nodes needed to disconnect the network. The obtained topology is shown in Figure 11(b), achieving maximum robustness under area and power constraints.

Table 14. Summary of Topology Generation Methods

Topology Type	Cited Work	Key Method	Strengths	Limitations
Application-agnostic	[57]	ILP-based	Broad design space and holistic optimization	Suboptimal for specific applications
	[206]	RL-based	High fault-tolerance and robustness	High computational complexity
Application-specific	[62]	Bayesian optimization (Selection-based)	Application-specific performance and overhead	Limited design space
	[175]	Multi-objective optimization	Large design space and cost-effectiveness	High computational complexity
	[176]	Space-filling curves	Linear dataflow affinity and low latency	High specialization and limited applicability
	[73]	Constraint-aware generation	Physical constraint awareness and feasibility	Performance potentially limited by constraints
	[4]	Traffic-aware vertical link placement	Fewer vertical links and lower energy/delay	Traffic pattern dependency

For specific applications, inter-chiplet network topologies can be tailored to their unique communication characteristics, thereby enhancing performance while minimizing overhead. Existing solutions typically fall into two categories: selection-based or direct-generation methods. Selection-based methods choose from a set of predefined candidate topologies, such as Mesh and Torus. For instance, Hao et al. [62] propose a Bayesian optimization algorithm for tensor applications. They encode candidate topology indices into chromosomes and iteratively optimize them until an optimal solution is reached, as shown in Figure 11(c). In contrast, direct-generation methods explore a larger design space than selection-based approaches to identify potentially more optimal, irregular topologies. Sharma et al. [175], for example, introduce a generation algorithm for 2.5D IMC systems that considers the traffic characteristics of DNN workloads. They employ a multi-objective optimization algorithm that adjusts the maximum number of system links in the outer loop while concurrently modifying the inter-chiplet link distribution in the inner loop. The resulting topology, depicted in Figure 11(d), improves performance and energy efficiency with lower fabrication costs compared to the general-purpose Kite topology [10]. Sharma et al. [176] leverage space-filling curves to generate a specialized topology with continuous chiplet paths that align with the linear dataflow of CNNs, resulting in inter-layer communication characterized by single, low-latency hops between neighboring chiplets. Additionally, Iff et al. [73] incorporate physical constraints by limiting the maximum link lengths while generating topologies. For 3D network topology generation, Agyeman et al. [4] introduce an algorithm to reduce the number of vertical links while maintaining NoC performance. Their approach first simulates the target application on an initial 3D Mesh network to determine the average utilization of all potential vertical links. Based on these utilization data, they construct the inter-chiplet network topology by strategically placing a designer-specified number of vertical connections at the inter-layer locations with the highest traffic. The resulting topology, as shown in Figure 11(e), achieves lower energy consumption and a significant reduction in packet delay compared to the homogeneous 3D Mesh.

The emergence of workloads such as heterogeneous computing has spurred the demand for application-specific topologies. These tailored designs can significantly mitigate communication bottlenecks to achieve superior performance under similar area and power constraints. Meanwhile, general topologies remain essential, providing a robust and reliable foundation for chiplet systems with simpler traffic patterns. These complementary approaches are advancing inter-chiplet networks toward more intelligent and efficient designs. An important direction for future work is to

Table 15. Summary of Routing Generation Methods

Routing Type	Cited Work	Key Method	Strengths	Limitations
Application-agnostic	[57]	ILP-based	Minimizes maximum channel load and achieves deadlock freedom	Requires known global topology
	[217]	Node abstraction	Modular (no global topology), system-level deadlock freedom	Suboptimal (uses agnostic metrics)
	[17]	Heuristic-based router presorting	Improved search efficiency	Suboptimal (uses agnostic metrics)
Application-specific	[216]	Traffic-aware turn prohibition	Application-aware, increased path diversity and network throughput	Less general and workload-dependent

incorporate physical constraints, such as wiring density and thermal hotspots, into the topology generation process, ensuring that logically optimal designs remain physically realizable in dense 3D packages.

4.3.2 Routing Generation. Table 15 summarizes several existing routing algorithms and highlights their respective advantages and disadvantages. Expert-designed routing solutions are often restricted to specific topologies [79, 155, 227], and can result in suboptimal latency and throughput. To address these limitations, the NetSmith [57] framework automatically generates routing by first enumerating all shortest paths between node pairs. It then employs an ILP-based formulation to select a subset of these paths, aiming at minimizing the maximum channel load. In addition to performance, deadlock-free routing is necessary unless specific flow control [134] or recovery [212] mechanisms are employed. While NetSmith achieves deadlock freedom using escape virtual channels, its generation process implicitly relies on a known inter-chiplet network topology. This assumption does not hold for scenarios where chiplets from different vendors are designed independently, often without prior knowledge of the complete system. To address this, Yin et al. [217] propose the modular routing algorithm. For each chiplet, this algorithm abstracts the rest of the system as a single virtual node and searches for optimal boundary router locations and prohibited turns at the chiplet-interposer interface. This method ensures an acyclic channel dependency graph, thereby preventing system-level deadlocks. The efficiency of this algorithm can be improved by presorting routers with heuristic rules during the search [17]. The above approaches primarily use application-agnostic metrics for determining these critical turn restrictions. Recognizing that performance could be further enhanced by considering workload characteristics, Ye et al. [216] propose an application-specific deadlock-free routing method. It builds upon the modular routing by incorporating traffic distribution information from applications into the selection of prohibited turns. Such an application-aware strategy results in fewer prohibited turns compared to application-agnostic ones, leading to increased routing path diversity and improved network throughput.

Inter-chiplet routing introduces a hierarchical design challenge not present in monolithic systems, as system-level channel dependencies are now influenced by the internal structures of each chiplet. Consequently, the primary design objective is to manage the tradeoff between ensuring deadlock-free operation and optimizing latency and throughput. Future work may focus on lightweight, adaptive algorithms that help mitigate runtime congestion and thermal throttling.

4.4 Reusable Package Design

Reusable packages are pivotal for agile chiplet integration, with their performance and usability determined by two key aspects: architecture and automated configuration.

Table 16. Summary of Reusable Package Architectures

Work	Packaging Technology ¹	Allocation Granularity	Configurable Network	Cost	Scalability	Comm. Latency ²	App. Domain	EDA Tool Support
Active SISL [185]	Active SI	Tile	✗	High	Medium	High	General	✗
Passive SISL [185]	Passive SI	Tile	✗	Low	High	High	General	✗
FSMC [9]	OS	Socket	✗	Low	High	High	General	✗
ZGlue [145]	Active SI	Tile	✓	High	Medium	Low	General	✓
Sipterposer [38]	Passive SI	Tile	✓	Low	High	Medium	General	✓
Neksus [54]	Passive SI + network die	N/A ³	✗	Medium	Low	High	General	✗
Adapt NoC [225]	Active SI	Tile	✓	High	Medium	Low	General	✗
NoI for AI [200]	Active SI	Tile	✓	High	Medium	Low	DNN	✗
PBS Interposer [184]	Passive SI	Socket	✗	Low	High	Medium	General	✗
EMAB [219]	Active bridge	Tile	✓	Medium	High	Low	General	✗
Active GIA [108]	Active SI	Tile	✓	High	Medium	Low	General	✓
Passive GIA [108]	Passive SI	Tile	✓	Low	High	Medium	General	✓
FPIA [82]	Active SI	Tile	✓	High	Medium	Low	General	✓
LUCIE [114]	Passive SI	Tile	✓	Low	High	Medium	General	✓
SHINSAI [83]	Active SI	Tile	✓	High	Medium	Low	General	✓
DSRI [125]	Active SI	Tile	✓	High	Medium	Low	Radar /Sonar	✓
Passive Silicon [183]	Passive SI	Tile	✓	Low	High	Medium	General	✗

¹ In this column, SI and OS denote silicon interposer and organic substrate, respectively.

² Inter-chiplet communication latency is influenced by both network configurability and the type of package (i.e., active or passive).

³ Given that Nexsus connects chiplets to the peripheral interfaces of its network die, its resource allocation granularity is neither socket-based nor tile-based.

4.4.1 Reusable Package Architecture. An ideal reusable package must address two fundamental demands: enabling the flexible placement of variably sized chiplets and accommodating diverse application-specific communication requirements. Success hinges on three dimensions: packaging technology, resource allocation granularity, and the configurability of the inter-chiplet network. Table 16 summarizes the characteristics of representative architectures.

Reusable packaging technologies are broadly classified as either active or passive. Active packages, such as active interposers and active bridges, integrate components like repeaters directly. This enhances data transmission by reducing interconnect delay and enabling higher network frequencies [187]. It also simplifies the implementation of configuration logic (e.g., switches) for configurable inter-chiplet interconnection, thereby avoiding the intricate configuration process required for

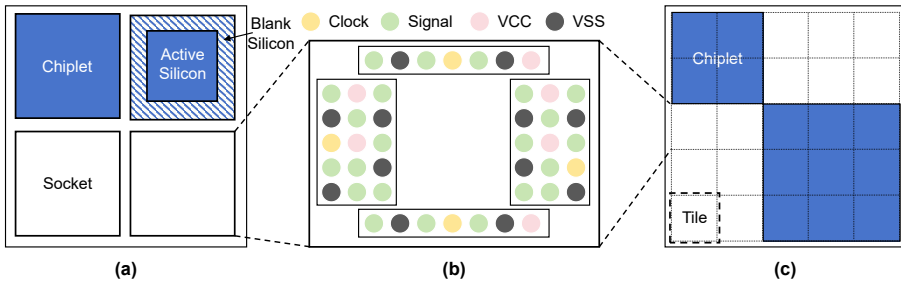


Fig. 12. Illustration of resource allocation granularity: (a)/(c) socket-/tile-based allocation, and (b) an example microbump map for a socket or a tile.

passive alternatives [38, 108]. However, these benefits are offset by significantly higher costs from complex FEOL processes and scalability constraints imposed by reticle limits. While solutions like Nexsus aim at reducing cost by replacing a full active interposer with smaller network dies, their limited number of peripheral D2D interfaces severely constrains system scalability [54]. In contrast, passive packages do not embed active components. Although they may have comparatively lower inter-chiplet communication performance and more complicated configuration processes, passive solutions offer superior cost-effectiveness. Furthermore, advanced manufacturing techniques like lithography stitching [69] enable the creation of large-area passive interposers, greatly enhancing scalability. While developed primarily for 2D or enhanced-2D systems, these solutions can be readily extended to 3D architectures [185, 219].

Resource allocation granularity is the smallest unit of signals assignable to a chiplet, encompassing inter-chiplet communication, external I/O, power, and clock functions. As illustrated in Figure 12(a)/(c), existing reusable packages are categorized into socket-based and tile-based allocation strategies. The socket-based approach dedicates an entire socket to a single chiplet, with each socket providing a standard set of resource signals shown in Figure 12(b). For example, each socket of the PBS interposer includes microbumps for SerDes, GPIO, and PCIe to support inter-chiplet and off-chip data transmission, as well as a modular power template for logic and SRAM. A major drawback of this strategy is the requirement that a chiplet's size must precisely match the socket dimension. If a chiplet's active area is smaller, it needs additional blank silicon to fit the socket, increasing costs and potentially running contrary to fundamental silicon economics. In contrast, the tile-based strategy allows a single chiplet to span multiple smaller tiles, reducing the wasted chiplet area required for size matching. For example, each FPIA tile provides switches for configurable inter-chiplet interconnection, along with 256 data and 64 power/ground microbumps. To enable more flexible communication, LUCIE utilizes a refined tile architecture that incorporates two distinct groups of transceiver microbumps per edge. This design enables the flexible implementation of various communication patterns, such as direct neighbor and diagonal connections, by employing different interposer wiring schemes while maintaining a consistent microbump mapping for the chiplets. Although tile-based allocation strategies offer increased adaptability for diverse chiplet sizes, they also introduce greater complexity in chiplet design. Designers must carefully orchestrate the placement of IPs to ensure that the communication, power, and clock signal requirements within each chiplet tile do not exceed the capacity of the corresponding package tile.

Adapting an inter-chiplet network's topology and routing to match application-specific communication patterns is critical for performance. While routing can be easily configured through software-defined routing tables, achieving topological configurability presents a significant hardware challenge, with implementations differing markedly between active and passive packages.

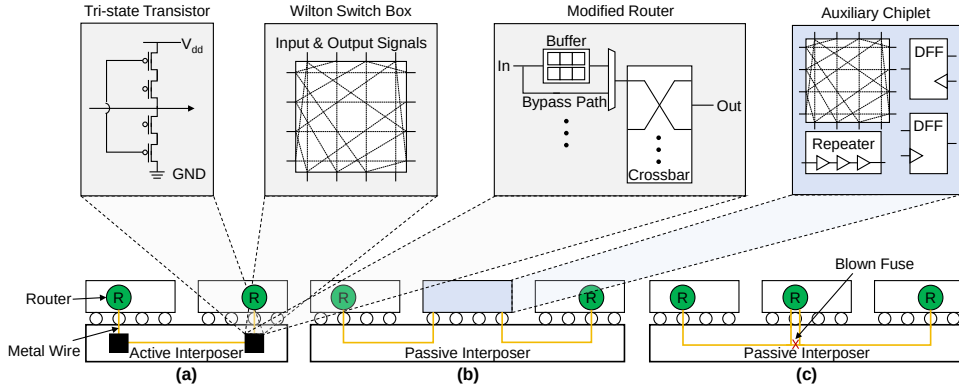


Fig. 13. Existing implementations of configurable inter-chiplet networks: (a) active logic embedded in the active package, (b) auxiliary chiplets, and (c) blown fuses.

In active packages, network configurability is realized through active logic embedded within the interposer or bridge. As shown in Figure 13(a), existing implementations include switches based on tri-state transistors [225], Wilton switch boxes [82, 83], and modified routers [108, 125]. Conversely, passive packages lack these active components and therefore require alternative strategies for flexible connections. One passive strategy utilizes interposers with disconnected metal wire segments that link only adjacent sockets/tiles, such as in the PBS interposer and LUCIE. This approach becomes problematic when system constraints, such as thermal management, force the placement of communicating chiplets on non-adjacent tiles [133]. To bridge this distance, auxiliary chiplets are employed [38, 108, 114]. As depicted in Figure 13(b), these specialized chiplets incorporate wires, retiming flip-flops for long-distance communication, or even programmable switches to establish the required connections. A second passive strategy used in Sipterposer and passive GIA builds the interposer with long wires that span multiple tiles. This architecture, however, is susceptible to signal interference when multiple chiplets simultaneously use the same wires. To isolate communication paths, a fuse-based approach is employed [38, 108], as shown in Figure 13(c). It uses a laser to break the long wires at specific points, creating dedicated links. However, this technique incurs considerable configuration costs and time, as the number of fuses that need to be blown can reach tens of thousands [114]. Furthermore, residual fuse material may act as an antenna, potentially compromising signal integrity and exacerbating crosstalk [114].

4.4.2 Reusable Package Configuration. As indicated in Table 16, reusable packages often incorporate automated configuration tools to accelerate the time-to-market for chip products. In these packages, the structures of D2D interfaces and interconnects are predefined, simplifying the configuration process to chiplet placement and signal mapping using configurable hardware.

The tile-based or socket-based structures of reusable packages naturally discretize the placement plane into grids. This allows established placement algorithms to be applied with minor modifications. For example, the GIA framework introduces a two-stage SA algorithm. It starts with a thermal optimization stage, followed by a holistic optimization stage that minimizes interposer power, communication latency, and peak system temperature. FPPIA aims at minimizing the estimated total wirelength, thereby reducing wire resource consumption and communication delay. It utilizes quadratic programming to obtain an initial placement with continuous coordinate values, which is then refined through an SA-based search. The subsequent signal mapping stage establishes designated connections between chiplets using available configurable logic and wire resources,

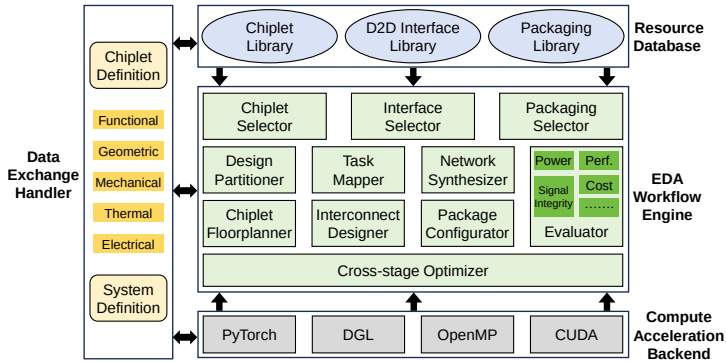


Fig. 14. Overview of the proposed unified chiplet design platform.

conceptually analogous to FPGA routing. Active GIA maps the generated inter-chiplet network topology onto the configurable communication fabric by first assigning network routers to specific tiles and constructing an interposer resource graph that details available wires and their penalties. Each network link is then allocated a dedicated and shortest path using a modified negotiated routing algorithm [137] applied to the interposer resource graph. For passive GIA, the mapping process further reduces the use of auxiliary chiplets by assigning them higher penalties. Other mapping methods [38, 82, 145] use different heuristics, such as a re-routing algorithm based on the maze method [82], to achieve similar goals. In contrast to the above solutions that perform placement and mapping separately, LUCIE introduces a combined approach. It uses subgraph monomorphism algorithms to map a problem graph, which encodes the required chiplet connections, onto an interposer graph that describes the physical layout and fixed wiring of the interposer. When a matched subgraph is found, both chiplet placement and mapping are obtained.

In summary, the development of reusable packages is driven by a fundamental tradeoff between performance and cost. Active architectures typically achieve higher performance and built-in configurability, which often comes with increased cost and scalability limitations. Passive solutions, in contrast, are generally more cost-effective and scalable, although they pose challenges for network configuration. In practice, the viability of these platforms increasingly relies on advances in EDA tools that support automated chiplet placement and signal mapping, enabling more efficient system integration.

5 Future Research Directions

Figure 14 illustrates the unified chiplet design platform we are developing, which consists of four primary components:

- **Resource Database:** Contains descriptions of in-house and third-party chiplets, D2D interfaces, and packaging technologies.
- **EDA Workflow Engine:** Orchestrates the entire design process, encompassing both top-down and bottom-up stages detailed in Sections 3 and 4.
- **Compute Acceleration Backend:** Leverages both deep learning frameworks (e.g., PyTorch [161] and DGL [201]) for data-driven acceleration and high-performance computing libraries (e.g., OpenMP [30] and CUDA [52]) for parallel computing on multi-core processors and GPUs.
- **Data Exchange Handler:** Manages data format conversions by handling definitions of individual chiplets and entire systems from perspectives such as functional and geometric. It ensures tool interoperability and facilitates cross-stage co-optimization.

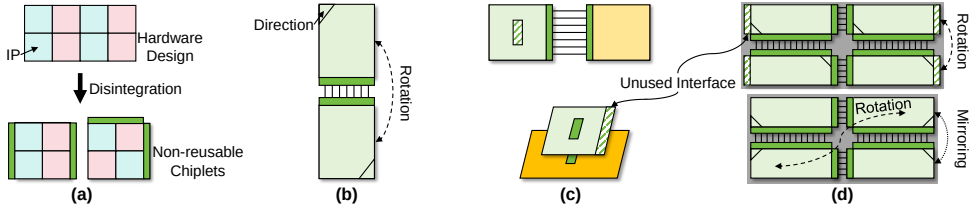


Fig. 15. Constraints and strategies for chiplet reuse. (a) Inconsistent floorplans disable chiplet mask reuse. (b) Chiplet rotation facilitates the reuse of parallel interfaces. (c) Adding redundant interfaces can enhance chiplet reusability. (d) Mirroring a chiplet layout can reduce the number of required D2D interfaces.

By flexibly combining and reusing common modules, our platform supports top-down, bottom-up, and hybrid design methodologies, thereby streamlining chiplet development and reducing toolchain complexity. Our experience in developing this platform highlights that despite significant advancements in chiplet design automation, tremendous challenges remain in unleashing the full potential of chiplet-based integration. We have identified the following key challenges:

Challenge 1: Toward reuse-aware decomposition in the top-down design flow. As highlighted in Section 2.2, chiplet reuse is a powerful strategy for reducing NRE costs and time-to-market. However, existing top-down design flows primarily consider reuse during the partitioning stage [111, 170]. To fully leverage its potential, we need to reshape the entire design flow to address the unique constraints and expanded design space introduced by reuse. Several factors illustrate this complexity. First, physical reuse imposes strict floorplanning constraints. As shown in Figure 15(a), to share the same mask, reusable chiplets must maintain identical IP and D2D interface placements across all systems. Second, reuse complicates D2D interface assignment, which is crucial for minimizing area overhead. As depicted in Figure 15(b), for parallel interfaces with stringent communication distance requirements, chiplet rotation can be used to reduce the number of required interfaces. In other cases, inserting redundant D2D interfaces can also facilitate chiplet reuse [180]. Figure 15(c) shows that a chiplet incorporating both parallel and 3D interfaces can be reused across two different systems through interface multiplexing. Third, layout mirroring offers another optimization strategy [180]. As shown in Figure 15(d), mirroring can reduce area overhead by requiring fewer D2D interfaces than the placement solely based on chiplet rotation. This technique reuses the chiplet floorplan and incurs only additional mask costs, which are well-amortized in high-volume production. Table 17 quantitatively compares NRE and RE costs under different reuse strategies. Developing a reuse-aware top-down flow first requires effective strategies to select the optimal reuse method, such as deciding between an identical or a mirrored mask. This flow must then be able to find a globally optimal solution within the unique design space constrained by the reuse method choice. This task is challenging for a single design, and its complexity explodes when decomposing multiple designs simultaneously to maximize chiplet reuse. This transforms the problem into a large-scale combinatorial optimization task, where discrete reuse choices (e.g., identical vs. mirrored masks) are tightly coupled with continuous floorplanning variables. A key research question is: how to develop a scalable co-optimization framework (e.g., based on hierarchical optimization or ILP) that can efficiently navigate this mixed discrete-continuous design space to find a Pareto-optimal frontier balancing metrics like NRE cost and system performance?

Challenge 2: Bridging the semantic gap between application code and chiplet APIs in the bottom-up design flow. Agile integration is the key characteristic of the bottom-up design flow. As elaborated in Section 4.4, reusable packages and associated automated configuration tools aim at circumventing the lengthy design cycle of custom packages. However, this shifts the development bottleneck to the chiplet selection and task mapping stage. Chiplet libraries generally

Table 17. Summary of Costs for Different Chiplet Reuse Strategies

Reuse Strategy	Key Characteristics	Cost per System
Baseline (no reuse)	Each system requires a unique NRE cost	$C_{NRE} + C_{RE}$
Perfect mask reuse (Figure 15(b))	A single NRE cost is amortized over N systems	$C_{NRE}/N + C_{RE}$
Mask reuse with redundant D2D interfaces (Figure 15(c))	Higher amortized NRE cost and RE cost	$(1 + \alpha)C_{NRE}/N + (1 + \beta)C_{RE}$
Reuse with layout mirroring (Figure 15(d))	Higher amortized NRE cost and basic RE cost	$(1 + \gamma)C_{NRE}/N + C_{RE}$

C_{NRE} and C_{RE} are the NRE and RE costs of a basic non-reusable chiplet, respectively. N is the number of systems reusing the chiplet. α and β are the proportional NRE and RE costs introduced by added interfaces, respectively. γ is the proportional NRE cost caused by an additional mask set.

Table 18. Summary of Task Mapping Methods in Bridging the Semantic Gap

Cited Work	Key Characteristics	Semantic Mapping Capability
[108]	Neglect of semantic mapping	-
[11, 35]	Limited application domains and limited chiplet architectures	Low
[40, 107]	Automation for known patterns	Medium
[136]	High automation and flexibility	High

include a variety of heterogeneous chiplets invoked via specific APIs, while user applications consist of numerous functions that could potentially be accelerated by different chiplets. A significant challenge arises as users usually provide source code (e.g., C++, Python) rather than abstracted task graphs, creating a semantic gap between the user code and chiplet APIs. For instance, as shown in Figure 16(a), user code might implement **Matrix-Vector Multiplication (MVM)**, while an available NN accelerator chiplet only supports **Convolution (Conv2D)**. Manually bridging this gap by rewriting code and transforming data layouts is feasible but extremely time-consuming and error-prone, especially with many chiplet and function types, fundamentally undermining the agility of the bottom-up methodology. Existing task mapping research often ignores this semantic mapping problem [108] or addresses it narrowly for specific application domains and limited chiplet architectures [11, 35]. Two promising automated approaches are emerging. The first utilizes string-based automata [40, 107]. As shown in Figure 16(b), it requires users to initially define target function patterns. Pattern-matching algorithms then identify these patterns in the user code and replace them with corresponding API-invoking code. Drawbacks of this method include the laborious effort required to define numerous function patterns and its inability to handle unforeseen code structures. An alternative approach, as depicted in Figure 16(c), leverages LLMs [136]. This emerging direction, treating the problem as semantic translation, is part of a broader trend discussed in detail in Challenge 6. Furthermore, ensuring the correctness of code matching and replacement is crucial. This likely requires integration with traditional formal verification or simulation-based validation to prevent functional errors. Table 18 compares the effectiveness of the above task mapping methods. The primary technical barrier for any automated method is the significant runtime overhead of full validation, which conflicts with the agility. A key research question is: how to design fast verification techniques with high accuracy?

Challenge 3: Toward the cross-layer co-optimization with larger design space. While both top-down and bottom-up chiplet design flows aim at optimizing parameters across multiple

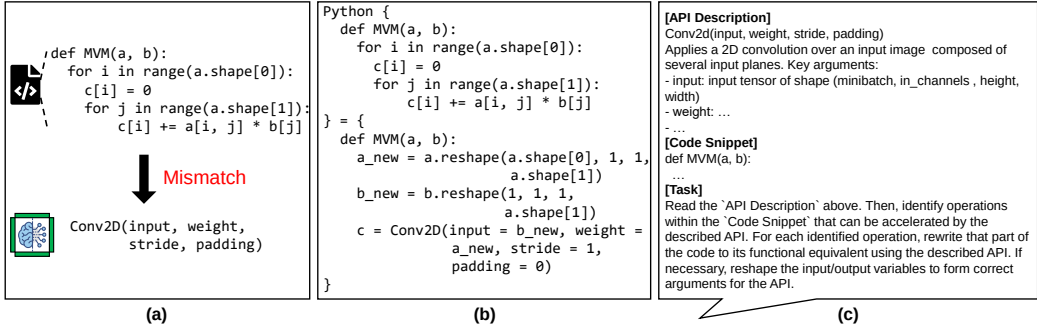


Fig. 16. Semantic gap and existing solutions. (a) Semantic mismatch between user code and chiplet API. (b) Pattern-based code transformation. (c) LLM-based code transformation.

layers, current research explores only a limited portion of this vast design space, leaving significant optimization opportunities untapped. Comprehensive cross-layer optimization is hindered by several key limitations. First, the selection of packaging technology receives inadequate consideration. Different packaging technologies present unique tradeoffs among cost, interconnect density, and thermal dissipation. While Shim et al. [177] propose a decision tree-based approach, its heavy reliance on expert-designed rules renders the selection process primarily qualitative. Hao et al. [62] use Bayesian optimization to select optimal solutions from candidate technologies. However, the vast number of available options makes this approach inefficient, likely requiring a combination with rule-based pruning. Furthermore, these methods are restricted to homogeneous packaging. As multi-chiplet systems scale, future designs may require heterogeneous packaging that integrates multiple technologies within a single package to meet diverse inter-chiplet communication requirements while concurrently optimizing overall cost and thermal performance [126]. Emerging concepts like 3.5D integration combining 2.5D and 3D packaging [180] illustrate this trend. Consequently, strategies for optimizing packaging configurations while considering their downstream impact on partitioning, floorplanning, and routing remain underdeveloped. Second, the automated selection of manufacturing process nodes for different chiplets during design partitioning is often neglected. Although current tools can accept user-specified node information [111, 228], they typically lack the capability to explore these choices automatically. This limitation hinders the ability to leverage heterogeneous process technologies for potential cost reduction. Third, detailed D2D interface planning, a critical step for system realization, is often overlooked. Existing research commonly assumes that distant chiplets can be directly connected on passive interposers [20, 65]. However, this assumption only holds for serial interfaces or low-frequency parallel interfaces, and fails for high-performance parallel D2D interfaces requiring close proximity (e.g., less than 2 mm [120]), necessitating placement algorithms that are aware of this communication constraint. Additionally, selecting optimal D2D interface types (e.g., serial versus parallel) to balance power, latency, and area remains a significant challenge. Finally, the scope of co-optimization could be extended even further to encompass the algorithm layer. When designing a specialized system like a DNN accelerator, for example, model parameters such as layer structures and inter-layer connectivity could be co-optimized within the broader hardware design space. The core technical barriers are the mix of discrete (e.g., process node, package type) and continuous (e.g., floorplan) variables. A specific research question is: How to formulate a unified representation for this heterogeneous design space that can formally capture the complex inter-dependencies between discrete choices (e.g., package type), continuous variables (e.g., floorplan geometry), and abstract goals (e.g., algorithm parameters), enabling a truly holistic optimization model?

Table 19. Summary of Time Overheads of Various Evaluation Stages in Design Flows

Cited Work	Evaluation Stage	Typical Evaluation Time
[111]	Early-stage partitioning	Minutes to hours
[57]	Inter-chiplet network	Minutes to days
[146]	Signal integrity	Hours to weeks
[160]	Thermal analysis	Hours to weeks
[228]	Holistic co-design	Hours to weeks

Challenge 4: Toward the cross-layer co-optimization with higher efficiency. Both top-down and bottom-up design flows require searching for optimal parameters within a vast design space, guided by evaluation models. Two key directions are essential to accelerate this process: reducing evaluation time per design point and enhancing search algorithm efficiency for rapid convergence to the optimal solution. As summarized in Table 19, the evaluation time for different stages of chiplet design can vary dramatically, from minutes to weeks, underscoring the importance of acceleration techniques. Efforts to accelerate evaluations include: (1) developing lightweight analytical models, such as the abstracted thermal model [164], (2) exploiting inherent parallelism in evaluation algorithms, like the multi-threaded inter-chiplet network simulation [45], and (3) employing data-driven methods, such as using an RNN for eye diagram prediction [146]. Efforts to improve search algorithm efficiency include: (1) white-box optimization, which leverages mathematical properties of the design space, like gradient descent-based placement [202], and (2) black-box optimization, such as simulated annealing. An emerging trend involves RL-based methods, which learn from historical data to guide the search process and avoid starting from scratch. Beyond accelerating individual optimization stages, efficiently co-optimizing multiple stages is also crucial. Different stages of the design flow are typically tightly coupled, with subsequent stages heavily dependent on the outcomes of preceding ones. This complicates dividing the optimization flow into smaller independently solvable sub-problems, creating expensive nested optimization loops. To mitigate this, current strategies attempt to prune unpromising solutions at early stages. For example, Li et al. [111] discard partitions if their inter-chiplet bandwidth requirements exceed feasible limits. Similarly, Coskun et al. [29] prune solutions based on partial evaluations of the objective function. In the future, these static rules could be replaced by more sophisticated data-driven decision models. A key research opportunity lies in using high-fidelity surrogate models. A specific research question is: how to design these models to accurately and rapidly predict final post-implementation metrics (e.g., IR drop, timing closure) from an early abstract design representation (e.g., a partitioned design), thus enabling high-confidence early-stage pruning? While a full-custom toolchain might offer maximum flexibility for implementing these optimization techniques, practical design flows often rely on integrating existing EDA tools. This reliance limits implementation flexibility and introduces new complexities, such as interpreting diverse and potentially indirect log files discussed in Challenge 5.

Challenge 5: Toward the seamless integration of existing tools into the design flow. Developing an entire EDA toolchain for multi-chiplet systems from scratch is largely impractical from an engineering perspective. A more feasible approach involves maximizing the reuse of existing commercial and open-source tools. However, this strategy introduces several hurdles. A primary challenge lies in adapting to existing tools, which often have compatibility issues or inherent limitations in meeting the design constraints of multi-chiplet systems. This challenge stems from the fact that most current EDA tools were developed for monolithic chips, lacking native support for system-level co-optimization across multiple chiplets, diverse packaging technologies, and heterogeneous process nodes. Addressing this requires careful configuration tweaks and

specific pre-processing steps. For example, the pseudo-3D design flow that utilizes 2D design tools requires scaling down the standard cell sizes [198]. Similarly, Sumbul et al. [189] propose a hierarchical method that treats individual dies as large macros integrated within a top-level wrapper. This abstraction enables the use of conventional 2D EDA tools for physical implementation and timing closure. Another significant hurdle is the effective incorporation of diverse third-party tools into the cross-layer optimization flow. Many attempts [86, 96] remain suboptimal because they resort to a stage-by-stage integration, which limits the potential for holistic optimization. A more desirable strategy, as demonstrated in [87], performs the iterative refinement of chiplet design constraints guided by the outputs of commercial parasitic extractors. Beyond directly utilizing the primary outputs of these tools, harnessing the rich information within their reports and log files can further enhance design flow efficiency. Table 20 summarizes this complex data landscape, highlighting the mix of structured, semi-structured, and unstructured outputs generated throughout a typical chip design flow. Some initial efforts have been made. For example, AutoDSE [182] employs a rule-based approach, leveraging performance reports to resolve bottlenecks in high-level synthesis. Another example is ChipGPT-FT [14], which uses an LLM-based method that analyzes Verilog compiler outputs to enhance the correctness of generated hardware code. However, these approaches are typically limited to specific design stages. Therefore, applying this rich, cross-stage data for comprehensive and automated co-optimization remains a critical and emerging area for research. The use of advanced data-driven techniques, particularly LLMs, for this task is detailed in Challenge 6. Furthermore, this integration challenge highlights the limitations of existing EDA ecosystems regarding compatibility, as the lack of standardized data formats and APIs for exchanging design intent and analysis results between tools from different vendors severely impedes interoperability. Future directions should include not only advanced co-optimization algorithms but also significant efforts toward toolchain extension and standardization. Promising initiatives, such as TSMC's 3Dblox standard [1], are emerging to address this by modularizing the design flow and standardizing interfaces to foster a more open ecosystem. Complementing this, the Open Compute Project's **Chiplet Design Exchange Markup Language (CDXML)** [150] focuses on standardizing the upstream exchange of chiplet models, including thermal, mechanical, and I/O specifications. However, these standards are in early stages and currently focus more on physical implementation rather than encompassing the full spectrum of cross-layer co-optimization. Building on this, the community needs to establish comprehensive open standards (e.g., extensions to existing formats like LEF/DEF or new data formats for functional descriptions) to create a seamless and extensible chiplet design ecosystem. A key research question is: what is the minimal formal data model required to represent and exchange cross-layer design intent (e.g., design partitioning and I/O optimization) between tools? Developing an open-standard data model for the complete design state that is far richer than 3Dblox's physical focus is a critical step toward true interoperability.

Challenge 6: Toward the LLM-native chiplet design automation. The potential of LLMs in EDA extends far beyond the semantic mapping (Challenge 2) and log analysis (Challenge 5) tasks. We envision a future moving toward LLM-native design automation, where LLMs act as a central controller for the entire workflow. This presents three-tiered research challenges. First, enhancing LLMs to understand high-level design intent (e.g., minimize latency between CPU and accelerator) and perform cross-stage root-cause analysis by correlating heterogeneous data from different tools (e.g., linking a timing violation report to a thermal hotspot log). Second, moving to generative hardware design, which involves generating PPA-aware and verifiable RTL or HLS code from natural language specifications. The core research hurdles here are ensuring functional correctness and preventing hardware hallucinations (i.e., syntactically correct but functionally flawed or non-synthesizable code). Third, exploring LLMs as a novel optimization engine to navigate the vast

Table 20. Summary of Data Characteristics From Representative EDA Tools in Design Flows

Design Stage	Example Tool	Primary Output	Data Characteristics (Structure, Format, Size)
High-level synthesis	Cadence Stratus	Verilog/VHDL, performance reports	Semi-structured (HDL, reports), unstructured (logs), MBs
Logic synthesis	Synopsys DC	Gate-level netlist, timing/area reports	Semi-structured (Verilog, reports), unstructured (logs), MBs to GBs
Physical implementation	Cadence Innovus	Layout, parasitics, log files	Structured (GDSII, DEF, SPEF), unstructured (logs), GBs
Multi-physics analysis	Ansys RedHawk	Thermal/Voltage maps, reports	Structured (grid formats), semi-structured (reports), unstructured (logs), GBs
Verification	Mentor Calibre	Error reports, parasitic netlists	Semi-structured (reports, netlists), unstructured (logs), GBs

chiplet design space (Challenge 3). A key research question is: can an LLM, trained on thousands of past designs, learn the intuition to directly generate a near-optimal system configuration (e.g., package type, process nodes, partitioning) without an exhaustive algorithmic search?

6 Conclusion

Chiplet technology has emerged as a pivotal solution in the post-Moore era. However, its adoption introduces a vast and intricate design space, demanding sophisticated EDA tools to identify optimal solutions. To this end, this article begins by introducing the foundational concepts, technical characteristics, and evaluation models of chiplets. It then presents a user-centric and methodology-driven survey of advancements in design automation, structured around the orthogonal top-down and bottom-up paradigms. Finally, the article discusses key challenges and potential future research directions. Addressing these challenges is crucial for enabling more agile, high-performance, and cost-effective chiplet-based integration.

References

[1] 2024. 3Dblox– Chiplet Connectivity and Physical Properties Description Language. Retrieved February 21, 2026 from <https://sagroups.ieee.org/3537/>

[2] 2024. Chapter 22: Interconnects for 2D and 3D Architectures. Retrieved February 21, 2026 from https://eps.ieee.org/wp-content/uploads/2025/11/HIR_2024_ch22_2D-3D.pdf

[3] Niket Agarwal, Tushar Krishna, Li-Shiuan Peh, and Niraj K. Jha. 2009. GARNET: A detailed on-chip network model inside a full-system simulator. In *Proceedings of the 2009 IEEE International Symposium on Performance Analysis of Systems And Software*. IEEE, 33–42.

[4] Michael Opoku Agyeman, Ali Ahmadinia, and Nader Bagherzadeh. 2015. Performance and energy aware inhomogeneous 3d networks-on-chip architecture generation. *IEEE Transactions on Parallel and Distributed Systems* 27, 6 (2015), 1756–1769.

[5] Mudasir Ahmad, Javi DeLaCruz, and Anu Ramamurthy. 2022. Heterogeneous integration of chiplets: Cost and yield tradeoff analysis. In *Proceedings of the 2022 23rd International Conference on Thermal, Mechanical and Multi-Physics Simulation and Experiments in Microelectronics and Microsystems*. IEEE, 1–9.

[6] Charles Alpert and Anirudh Devgan. 1997. Wire segmenting for improved buffer insertion. In *Proceedings of the 34th Annual Design Automation Conference*. 588–593.

[7] Akhil Arunkumar, Evgeny Bolotin, Benjamin Cho, Ugljesa Milic, Eiman Ebrahimi, Oreste Villa, Aamer Jaleel, Carole-Jean Wu, and David Nellans. 2017. MCM-GPU: Multi-chip-module GPUs for continued performance scalability. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 320–332.

[8] Sabyasachee Banerjee and Subhashis Majumder. 2014. A thermal aware 3D IC partitioning technique. In *18th International Symposium on VLSI Design and Test*. IEEE, 1–6.

[9] Noah Beck, Sean White, Milam Paraschou, and Samuel Naffziger. 2018. ‘Zeppelin’: An SoC for multichip architectures. In *Proceedings of the 2018 IEEE International Solid-State Circuits Conference*. IEEE, 40–42.

- [10] Srikant Bharadwaj, Jieming Yin, Bradford Beckmann, and Tushar Krishna. 2020. Kite: A family of heterogeneous interposer topologies enabled via accurate interconnect modeling. In *Proceedings of the 2020 57th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [11] Jingwei Cai, Zuocong Wu, Sen Peng, Yuchen Wei, Zhanhong Tan, Guiming Shi, Mingyu Gao, and Kaisheng Ma. 2024. Gemini: Mapping and architecture co-exploration for large-scale dnn chiplet accelerators. In *Proceedings of the 2024 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 156–171.
- [12] Anthony Chan Carusone, Behzad Dehlaghi, Rudy Beerkens, and Davide Tonietto. 2016. Ultra-short-reach interconnects for package-level integration. In *Proceedings of the 2016 IEEE Optical Interconnects Conference*. IEEE, 10–11.
- [13] Kyungwook Chang, Shidhartha Das, Saurabh Sinha, Brian Cline, Greg Yeric, and Sung Kyu Lim. 2019. System-level power delivery network analysis and optimization for monolithic 3-D ICs. *IEEE Transactions on Very Large Scale Integration Systems* 27, 4 (2019), 888–898.
- [14] Kaiyan Chang, Kun Wang, Nan Yang, Ying Wang, Dantong Jin, Wenlong Zhu, Zhirong Chen, Cangyuan Li, Hao Yan, Yunhao Zhou, et al. 2024. Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [15] Wei-Ju Chang and Ruey-Beei Wu. 2018. Eye diagram estimation and equalizer design method for PAM4. In *Proceedings of the 2018 IEEE 22nd Workshop on Signal and Power Integrity*. IEEE, 1–4.
- [16] Muhammad Waqas Chaudhary, Andy Heinig, and Bhaskar Choubey. 2021. Holistic die-to-die interface design methodology for 2.5-D multichip-module systems. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 11, 12 (2021), 2171–2182.
- [17] Chixiao Chen, Jieming Yin, Yarui Peng, Maurizio Palesi, Wenxu Cao, Letian Huang, Amit Kumar Singh, Haocong Zhi, and Xiaohang Wang. 2022. Design challenges of intra-and inter-chiplet interconnection. *IEEE Design and Test* 39, 6 (2022), 99–109.
- [18] Hung-Ming Chen, Chu-Wen Ho, Shih-Hsien Wu, Wei Lu, Po-Tsang Huang, Hao-Ju Chang, and Chien-Nan Jimmy Liu. 2023. Reshaping system design in 3D integration: Perspectives and challenges. In *Proceedings of the 2023 International Symposium on Physical Design*. 71–77.
- [19] Liang Chen, Wentian Jin, and Sheldon X-D Tan. 2022. Fast thermal analysis for chiplet design based on graph convolution networks. In *Proceedings of the 2022 27th Asia and South Pacific Design Automation Conference*. IEEE, 485–492.
- [20] Shixin Chen, Shanyi Li, Zhen Zhuang, Su Zheng, Zheng Liang, Tsung-Yi Ho, Bei Yu, and Alberto L Sangiovanni-Vincentelli. 2023. Floorplet: Performance-aware floorplan framework for chiplet integration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 6 (2023), 1638–1649.
- [21] Shuangliang Chen, Saptadeep Pal, and Rakesh Kumar. 2024. Waferscale network switches. In *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture*. IEEE, 215–229.
- [22] Shixin Chen, Hengyuan Zhang, Zichao Ling, Jianwang Zhai, and Bei Yu. 2025. The survey of 2.5D integrated architecture: An EDA perspective. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASPDAC'25)*. Association for Computing Machinery, New York, NY, USA. 285–293.
- [23] Yen-Hao Chen, Po-Chen Huang, Fu-Wei Chen, Allen C-H Wu, and TingTing Hwang. 2019. Crosstalk-aware TSV-buffer insertion in 3D IC. In *Proceedings of the 2019 32nd IEEE International System-on-Chip Conference*. IEEE, 400–405.
- [24] Chia-Yuan Cheng, Shi-Yu Huang, Ding-Ming Kwai, and Yung-Fa Chou. 2017. DLL-assisted clock synchronization method for multi-die ICs. In *Proceedings of the 2017 IEEE International Conference on Computer Design*. IEEE, 473–476.
- [25] Chun-Han Chiang, Fu-Yu Chuang, and Yao-Wen Chang. 2020. Unified redistribution layer routing for 2.5 D IC packages. In *Proceedings of the 2020 25th Asia and South Pacific Design Automation Conference*. IEEE, 331–337.
- [26] Hong-Wen Chiou, Jia-Hao Jiang, Yu-Teng Chang, Yu-Min Lee, and Chi-Wen Pan. 2023. Chiplet placement for 2.5 D IC with sequence pair based tree and thermal consideration. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. 7–12.
- [27] Min-Hsuan Chung, Je-Wei Chuang, and Yao-Wen Chang. 2023. Any-angle routing for redistribution layers in 2.5 D IC packages. In *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [28] Jason Cong, Jie Wei, and Yan Zhang. 2004. A thermal-driven floorplanning algorithm for 3D ICs. In *Proceedings of the IEEE/ACM International Conference on Computer Aided Design*. IEEE, 306–313.
- [29] Ayse Coskun, Furkan Eris, Ajay Joshi, Andrew B. Kahng, Yenai Ma, Aditya Narayan, and Vaishnav Srinivas. 2020. Cross-layer co-optimization of network design and chiplet placement in 2.5-D systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 5183–5196.
- [30] Leonardo Dagum and Ramesh Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *IEEE computational science and engineering* 5, 1 (1998), 46–55.
- [31] Debendra Das Sharma, Gerald Pasdast, Sathya Tiagaraj, and Kemal Aygün. 2024. High-performance, power-efficient three-dimensional system-in-package designs with universal chiplet interconnect express. *Nature Electronics* 7, 3 (2024), 244–254.

- [32] Behzad Dehlaghi, Shayan Shahramian, Joshua Liang, Ryan Bepalko, Dustin Dunwell, James Bailey, Bo Wang, Alireza Sharif-Bakhtiar, Michael O'Farrell, Kerry Tang, et al. 2019. A 1.41-pJ/b 56-Gb/s PAM-4 receiver using enhanced transition utilization CDR and genetic adaptation algorithms in 7-nm CMOS. *IEEE Solid-State Circuits Letters* 2, 11 (2019), 248–251.
- [33] Xiao Dong, Songyu Sun, Yangfan Jiang, Jingtong Hu, Dawei Gao, and Cheng Zhuo. 2024. SPIRAL: Signal-power integrity co-analysis for high-speed inter-chiplet serial links validation. In *Proceedings of the 2024 29th Asia and South Pacific Design Automation Conference*. IEEE, 625–630.
- [34] Xiangyu Dong, Jishen Zhao, and Yuan Xie. 2010. Fabrication cost analysis and cost-aware design space exploration for 3-D ICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 29, 12 (2010), 1959–1972.
- [35] Yibo Du, Ying Wang, Bing Li, Fuping Li, Shengwen Liang, Huawei Li, Xiaowei Li, and Yinhe Han. 2024. Chiplever: Towards effortless extension of chiplet-based system for FHE. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [36] Gang Duan, Yosuke Kanaoka, Robin McRee, Bai Nie, and Rahul Manepalli. 2021. Die embedding challenges for EMIB advanced packaging technology. In *Proceedings of the 2021 IEEE 71st Electronic Components and Technology Conference*. IEEE, 1–7.
- [37] Yuanyuan Duan, Xingchen Liu, Zhiping Yu, Hanming Wu, Leilai Shao, and Xiaolei Zhu. 2024. RLPlanner: Reinforcement learning based floorplanning for chiplets with fast thermal analysis. In *Proceedings of the 2024 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1–2.
- [38] Pete Ehrett, Todd Austin, and Valeria Bertacco. 2019. SiPterposer: A fault-tolerant substrate for flexible system-in-package design. In *Proceedings of the 2019 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 510–515.
- [39] Pete Ehrett, Todd Austin, and Valeria Bertacco. 2021. Chopin: Composing cost-effective custom chips with algorithmic chiplets. In *Proceedings of the 2021 IEEE 39th International Conference on Computer Design*. IEEE, 395–399.
- [40] Vinicius Espindola, Luciano Zago, Hervé Yviquel, and Guido Araujo. 2023. Source matching and rewriting for MLIR using string-based automata. *ACM Transactions on Architecture and Code Optimization* 20, 2 (2023), 1–26.
- [41] Jia-Wei Fang and Yao-Wen Chang. 2008. Area-I/O flip-chip routing for chip-package co-design. In *Proceedings of the 2008 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 518–522.
- [42] Jia-Wei Fang, Chin-Hsiung Hsu, and Yao-Wen Chang. 2007. An integer linear programming based routing algorithm for flip-chip design. In *Proceedings of the 44th annual Design Automation Conference*. 606–611.
- [43] Jia-Wei Fang, I-Jye Lin, Ping-Hung Yuh, Yao-Wen Chang, and Jyh-Herng Wang. 2005. A routing algorithm for flip-chip design. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2005. IEEE, 753–758.
- [44] Jia-Wei Fang, Martin DF Wong, and Yao-Wen Chang. 2009. Flip-chip routing with unified area-I/O pad assignments for package-board co-design. In *Proceedings of the 46th Annual Design Automation Conference*. 336–339.
- [45] Yinxiao Feng and Kaisheng Ma. 2022. Chiplet actuary: A quantitative cost model and multi-chiplet architecture exploration. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 121–126.
- [46] Yinxiao Feng, Yuchen Wei, Dong Xiang, and Kaisheng Ma. 2024. Evaluating Chiplet-based {Large-Scale} interconnection networks via {Cycle-Accurate}{Packet-Parallel} simulation. In *Proceedings of the 2024 USENIX Annual Technical Conference*. 731–747.
- [47] Yinxiao Feng, Dong Xiang, and Kaisheng Ma. 2023. Heterogeneous die-to-die interfaces: Enabling more flexible chiplet interconnection systems. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 930–943.
- [48] Tomas Figliolia and Andreas G. Andreou. 2019. The conical-fishbone clock tree: A clock-distribution network for a heterogeneous chip multiprocessor AI chiplet. In *Proceedings of the 2019 22nd Euromicro Conference on Digital System Design*. IEEE, 160–165.
- [49] Bangqi Fu, Lixin Liu, Yang Sun, Wing-Ho Lau, Martin DF Wong, and Evangeline FY Young. 2024. Coplace: Coherent placement engine with layout-aware partitioning for 3d ics. In *Proceedings of the 2024 29th Asia and South Pacific Design Automation Conference*. IEEE, 65–70.
- [50] Marc Fyrbiak, Sebastian Wallat, Sascha Reinhard, Nicolai Bissantz, and Christof Paar. 2019. Graph similarity and its applications to hardware security. *IEEE Transactions on Computers* 69, 4 (2019), 505–519.
- [51] G. Gangasani, D. Hanson, D. Storaska, HH Xu, M. Kelly, M. Shannon, Michael Sorna, Michael Wielgos, PB Ramakrishna, S. Shi, et al. 2022. A 1.6 Tb/s chiplet over XSR-MCM channels using 113Gb/s PAM-4 transceiver with dynamic receiver-driven adaptation of TX-FFE and programmable roaming taps in 5nm CMOS. In *Proceedings of the 2022 IEEE International Solid-State Circuits Conference*. IEEE, 122–124.
- [52] Michael Garland, Scott Le Grand, John Nickolls, Joshua Anderson, Jim Hardwick, Scott Morton, Everett Phillips, Yao Zhang, and Vasily Volkov. 2008. Parallel computing experiences with CUDA. *IEEE Micro* 28, 4 (2008), 13–27.

- [53] Wilfred Gomes, Altug Koker, Pat Stover, Doug Ingerly, Scott Siers, Srikrishnan Venkataraman, Chris Pelto, Tejas Shah, Amreesh Rao, Frank O'Mahony, et al. 2022. Ponte Vecchio: A multi-tile 3D stacked processor for exascale computing. In *Proceedings of the 2022 IEEE International Solid-State Circuits Conference*. IEEE, 42–44.
- [54] Vidushi Goyal, Xiaowei Wang, Valeria Bertacco, and Reetuparna Das. 2020. Nexus: An interconnect for heterogeneous system-in-package architectures. In *Proceedings of the 2020 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 12–21.
- [55] Alexander Graening, Saptadeep Pal, and Puneet Gupta. 2023. Chiplets: How small is too small?. In *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [56] Alexander Graening, Jonti Talukdar, Saptadeep Pal, Krishnendu Chakrabarty, and Puneet Gupta. 2025. CATCH: A cost analysis tool for co-optimization of chiplet-based heterogeneous systems. Retrieved from <https://arxiv.org/abs/2503.15753>
- [57] Conor James Green and Mithuna Thottethodi. 2024. NetSmith: An optimization framework for machine-discovered network topologies. In *Proceedings of the 53rd International Conference on Parallel Processing*. 421–432.
- [58] David Greenhill, Ron Ho, David Lewis, Herman Schmit, Kok Hong Chan, Andy Tong, Sean Atsatt, Dana How, Peter McElheny, Keith Duwel, et al. 2017. 3.3 A 14nm 1GHz FPGA with 2.5 D transceiver integration. In *Proceedings of the 2017 IEEE International Solid-State Circuits Conference*. IEEE, 54–55.
- [59] Wenbo Guan, Xiaoyan Tang, Hongliang Lu, Yuming Zhang, and Yimen Zhang. 2023. ATT-TA: A cooperative multiagent deep reinforcement learning approach for TSV assignment in 3-D ICs. *IEEE Transactions on Very Large Scale Integration Systems* 31, 12 (2023), 1905–1917.
- [60] Yinhe Han, Haobo Xu, Meixuan Lu, Haoran Wang, Junpei Huang, Ying Wang, Yujie Wang, Feng Min, Qi Liu, Ming Liu, et al. 2024. The big chip: Challenge, model and architecture. *Fundamental Research* 4, 6 (2024), 1431–1441.
- [61] Cong Hao, Nan Ding, and Takeshi Yoshimura. 2016. An efficient algorithm for 3D-IC TSV assignment. In *Proceedings of the 2016 14th IEEE International New Circuits and Systems Conference*. IEEE, 1–4.
- [62] Xiaochen Hao, Zijian Ding, Jieming Yin, Yuan Wang, and Yun Liang. 2023. Monad: Towards cost-effective specialization for chiplet-based spatial accelerators. In *Proceedings of the 2023 IEEE/ACM International Conference on Computer Aided Design*. IEEE, 1–9.
- [63] Steven Herbst, Noah Moroze, Edgar Iglesias, and Andreas Olofsson. 2024. Switchboard: An open-source framework for modular simulation of large hardware systems. Retrieved from <https://arxiv.org/abs/2407.20537>
- [64] Takashi Hisada, Yasuharu Yamada, Junko Asai, and Toyohiro Aoki. 2012. Study of warpage and mechanical stress of 2.5 D package interposers during chip and interposer mount process. In *Proceedings of the International Symposium on Microelectronics*. International Microelectronics Assembly and Packaging Society, 000967–000974.
- [65] Yuan-Kai Ho and Yao-Wen Chang. 2013. Multiple chip planning for chip-interposer codesign. In *Proceedings of the 50th Annual Design Automation Conference*. 1–6.
- [66] Yang Hsu, Min-Hsuan Chung, Yao-Wen Chang, and Ci-Hong Lin. 2022. Transitive closure graph-based warpage-aware floorplanning for package designs. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–7.
- [67] Yang Hu, Xinhan Lin, Huizheng Wang, Zhen He, Xingmao Yu, Jiahao Zhang, Qize Yang, Zheng Xu, Sihuan Guan, Jiahao Fang, et al. 2024. Wafer-scale computing: Advancements, challenges, and future perspectives [Feature]. *IEEE Circuits and Systems Magazine* 24, 1 (2024), 52–81.
- [68] Ming-Yu Huang, Hung-Ming Chen, Kuan-Neng Chen, Shih-Hsien Wu, Yu-Min Lee, and An-Yu Su. 2020. A design flow for micro bump and stripe planning on modern chip-package co-design. In *Proceedings of the 2020 IEEE 70th Electronic Components and Technology Conference*. IEEE, 2236–2241.
- [69] PK Huang, CY Lu, WH Wei, Christine Chiu, KC Ting, Clark Hu, CH Tsai, SY Hou, WC Chiou, CT Wang, et al. 2021. Wafer level system integration of the fifth generation CoWoS®-S with high performance Si interposer at 2500 mm2. In *Proceedings of the 2021 IEEE 71st Electronic Components and Technology Conference*. IEEE, 101–104.
- [70] Shi-Yu Huang and Chih-Chieh Zheng. 2016. Die-to-die clock skew characterization and tuning for 2.5 D ICs. In *Proceedings of the 2016 IEEE 25th Asian Test Symposium*. IEEE, 221–226.
- [71] Patrick Iff, Maciej Besta, Matheus Cavalcante, Tim Fischer, Luca Benini, and Torsten Hoefer. 2023. Hexamesh: Scaling to hundreds of chiplets with an optimized chiplet arrangement. In *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [72] Patrick Iff, Benigna Bruggmann, Blaise Morel, Maciej Besta, Luca Benini, and Torsten Hoefer. 2025. RapidChiplet: A toolchain for rapid design space exploration of inter-chiplet interconnects. In *Proceedings of the 22nd ACM International Conference on Computing Frontiers (CF'25)*. 168–171.
- [73] Patrick Iff, Benigna Bruggmann, Maciej Besta, Luca Benini, and Torsten Hoefer. 2025. PlaceIT: Placement-based inter-chiplet interconnect topologies. Retrieved from <https://arxiv.org/abs/2502.01449>
- [74] Frank Imeson, Ariq Emtenan, Siddharth Garg, and Mahesh Tripunitara. 2013. Securing computer hardware using 3D integrated circuit ({{{IC}}}) technology and split manufacturing for obfuscation. In *Proceedings of the 22nd USENIX Security Symposium*. 495–510.

- [75] Randall Irwin, Krutikesh Sahoo, Saptadeep Pal, and Subramanian S. Iyer. 2021. Flexible connectors and PCB segmentation for signaling and power delivery in wafer-scale systems. In *Proceedings of the 2021 IEEE 71st Electronic Components and Technology Conference*. IEEE, 507–513.
- [76] Michael AB Jackson, Arvind Srinivasan, and Ernest S. Kuh. 1991. Clock routing for high-performance ICs. In *Proceedings of the 27th ACM/IEEE Design Automation Conference*. 573–579.
- [77] Abhi Jaiswal, KC Sharin Shahana, Sujitha Ravichandran, K. Adarsh, H. Bharath Bhat, Biresh Kumar Joardar, and Sumit K Mandal. 2024. HALO: Communication-aware heterogeneous 2.5 D system for energy-efficient LLM execution at edge. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 14, 3 (2024), 425–439.
- [78] Supreet Jeloka, Brian Cline, Shidhartha Das, Benoit Labbe, Alejandro Rico, Rainer Herberholz, Javier DeLaCruz, Rahul Mathur, and Shawn Hung. 2022. System technology co-optimization and design challenges for 3D IC. In *Proceedings of the 2022 IEEE Custom Integrated Circuits Conference*. IEEE, 1–6.
- [79] Natalie Enright Jerger, Ajaykumar Kannan, Zimo Li, and Gabriel H Loh. 2014. NoC architectures for silicon interposer systems: Why pay for more wires when you can get them (from your interposer) for free?. In *Proceedings of the 2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 458–470.
- [80] Nan Jiang, Daniel U. Becker, George Michelogiannakis, James Balfour, Brian Towles, David E. Shaw, John Kim, and William J. Dally. 2013. A detailed and flexible cycle-accurate network-on-chip simulator. In *Proceedings of the 2013 IEEE International Symposium on Performance Analysis of Systems and Software*. IEEE, 86–96.
- [81] Yande Jiang, Xu He, Chang Liu, and Yang Guo. 2015. An effective analytical 3D placer in monolithic 3D IC designs. In *Proceedings of the 2015 IEEE 11th International Conference on ASIC*. IEEE, 1–4.
- [82] Bo Jiao, Lei Xu, Xinyu Yu, Haitao Yang, Haozhe Zhu, Yu Wang, Jundong Zhu, Dexin Wen, Lingli Wang, Jun Tao, et al. 2024. FPIA: Communication-aware multi-chiplet integration with field-programmable interconnect fabric on reusable silicon interposer. *IEEE Transactions on Circuits and Systems I: Regular Papers* 71, 9 (2024), 4156–4168.
- [83] Bo Jiao, Haozhe Zhu, Yuman Zeng, Yongjiang Li, Jie Liao, Siyao Jia, Mochen Tian, Zexing Chen, Jundong Zhu, Dexin Wen, et al. 2025. 37.4 SHINSAI: A 586mm² reusable active TSV interposer with programmable interconnect fabric and 512Mb 3D underdeck memory. In *Proceedings of the 2025 IEEE International Solid-State Circuits Conference*. IEEE, 01–03.
- [84] Moongon Jung, Joydeep Mitra, David Z. Pan, and Sung Kyu Lim. 2011. TSV stress-aware full-chip mechanical reliability analysis and optimization for 3D IC. In *Proceedings of the 48th Design Automation Conference*. 188–193.
- [85] Moongon Jung, David Z Pan, and Sung Kyu Lim. 2012. Chip/package co-analysis of thermo-mechanical stress and reliability in TSV-based 3D ICs. In *Proceedings of the 49th Annual Design Automation Conference*. 317–326.
- [86] MD Arafat Kabir and Yarui Peng. 2020. Chiplet-package co-design for 2.5 D systems using standard ASIC CAD tools. In *Proceedings of the 2020 25th Asia and South Pacific Design Automation Conference*. IEEE, 351–356.
- [87] MD Arafat Kabir and Yarui Peng. 2021. Holistic chiplet–package co-optimization for agile custom 2.5-D design. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 11, 5 (2021), 715–726.
- [88] Ajaykumar Kannan, Natalie Enright Jerger, and Gabriel H. Loh. 2015. Enabling interposer-based disintegration of multi-core processors. In *Proceedings of the 48th international symposium on Microarchitecture*. 546–558.
- [89] Ramakrishna Kanungo, Swamynathan Siva, Nathaniel Bleier, Muhammad Husnain Mubarik, Lav Varshney, and Rakesh Kumar. 2023. Understanding interactions between chip architecture and uncertainties in semiconductor supply and demand. arXiv:2305.11059. Retrieved from <https://arxiv.org/abs/2305.11059>
- [90] Guruprasad Katti, Michele Stucchi, Kristin De Meyer, and Wim Dehaene. 2009. Electrical modeling and characterization of through silicon via for three-dimensional ICs. *IEEE transactions on Electron Devices* 57, 1 (2009), 256–262.
- [91] David Kehlet. 2029. Accelerating Innovation Through a Standard Chiplet Interface: The Advanced Interface Bus (AIB). Retrieved February 21, 2026 from <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/accelerating-innovation-%0A%0Athrough-aib-whitepaper.pdf>
- [92] Moein Khazraee, Lu Zhang, Luis Vega, and Michael Bedford Taylor. 2017. Moonwalk: Nre optimization in asic clouds. *ACM SIGARCH Computer Architecture News* 45, 1 (2017), 511–526.
- [93] Dae Hyun Kim, Krit Athikulwongse, and Sung Kyu Lim. 2009. A study of through-silicon-via impact on the 3D stacked IC layout. In *Proceedings of the 2009 International Conference on Computer-Aided Design*. 674–680.
- [94] Heegon Kim, Chunchun Sui, Kevin Cai, Bidyut Sen, and Jun Fan. 2017. Fast and precise high-speed channel modeling and optimization technique based on machine learning. *IEEE Transactions on Electromagnetic Compatibility* 60, 6 (2017), 2049–2052.
- [95] Jaeha Kim. 2014. Advanced modeling and simulation of state-of-the-art high-speed I/O interfaces. In *Proceedings of the IEEE 2014 Custom Integrated Circuits Conference*. IEEE, 1–122.
- [96] Jinwoo Kim, Gauthaman Murali, Heechun Park, Eric Qin, Hyoukjun Kwon, Venkata Chaitanya Krishna Chekuri, Nael Mizanur Rahman, Nihar Dasari, Arvind Singh, Minah Lee, et al. 2020. Architecture, chip, and package codesign flow for interposer-based 2.5-D chiplet integration enabling heterogeneous IP reuse. *IEEE Transactions on Very Large Scale Integration Systems* 28, 11 (2020), 2424–2437.

- [97] Tak-Yung Kim and Taewhan Kim. 2010. Bounded skew clock routing for 3D stacked IC designs: Enabling trade-offs between power and clock skew. In *Proceedings of the International Conference on Green Computing*. IEEE, 525–532.
- [98] Tak-Yung Kim and Taewhan Kim. 2011. Clock tree synthesis for TSV-based 3D IC designs. *ACM Transactions on Design Automation of Electronic Systems* 16, 4 (2011), 1–21.
- [99] Gokul Krishnan, A. Alper Goksoy, Sumit K. Mandal, Zhenyu Wang, Chaitali Chakrabarti, Jae-sun Seo, Umit Y Ogras, and Yu Cao. 2022. Big-little chiplets for in-memory acceleration of dnnns: A scalable heterogeneous architecture. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [100] Gokul Krishnan, Sumit K. Mandal, Manvitha Pannala, Chaitali Chakrabarti, Jae-Sun Seo, Umit Y. Ogras, and Yu Cao. 2021. SIAM: Chiplet-based scalable in-memory acceleration with mesh for deep neural networks. *ACM Transactions on Embedded Computing Systems* 20, 5s (2021), 1–24.
- [101] Bon Woong Ku, Kyungwook Chang, and Sung Kyu Lim. 2018. Compact-2D: A physical design methodology to build commercial-quality face-to-face-bonded 3D ICs. In *Proceedings of the 2018 International Symposium on Physical Design*. 90–97.
- [102] John H Lau. 2023. *Chiplet Design and Heterogeneous Integration Packaging*. Springer.
- [103] Chung-Chia Lee and Yao-Wen Chang. 2023. Floorplanning for embedded multi-die interconnect bridge packages. In *Proceedings of the 2023 IEEE/ACM International Conference on Computer Aided Design*. IEEE, 1–8.
- [104] Hsu-Chieh Lee, Yao-Wen Chang, and Po-Wei Lee. 2010. Recent research development in flip-chip routing. In *Proceedings of the 2010 IEEE/ACM International Conference on Computer-Aided Design*. 404–410.
- [105] Po-Wei Lee, Chung-Wei Lin, Yao-Wen Chang, Chin-Fang Shen, and Wei-Chih Tseng. 2009. An efficient pre-assignment routing algorithm for flip-chip designs. In *Proceedings of the 2009 IEEE/ACM International Conference on Computer-Aided Design*. 239–244.
- [106] Young-Joon Lee and Sung Kyu Lim. 2012. Fast delay estimation with buffer insertion for through-silicon-via-based 3D interconnects. In *Proceedings of the 13th International Symposium on Quality Electronic Design*. IEEE, 228–335.
- [107] Cangyuan Li, Ying Wang, Huawei Li, and Yinhe Han. 2023. APPEND: Rethinking ASIP synthesis in the era of AI. In *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [108] Fuping Li, Ying Wang, Yuanqing Cheng, Yujie Wang, Yinhe Han, Huawei Li, and Xiaowei Li. 2022. GIA: A reusable general interposer architecture for agile chiplet integration. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [109] Fuping Li, Ying Wang, Cheng Liu, Huawei Li, and Xiaowei Li. 2022. Noception: a fast ppa prediction framework for network-on-chips using graph neural network. In *Proceedings of the 2022 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1035–1040.
- [110] Fuping Li, Ying Wang, Meixuan Lu, Yutong Zhu, Haoran Wang, Zhun Zhao, Junpei Huang, Xiaotong Wei, Xihao Liang, Yujie Wang, et al. 2024. The decomposition and combination paradigms of chiplet-based integrated chips. *Integrated Circuits and Systems* 1, 1 (2024), 18–30.
- [111] Fuping Li, Ying Wang, Yujie Wang, Mengdi Wang, Yinhe Han, Huawei Li, and Xiaowei Li. 2024. Chipletizer: Repartitioning socs for cost-effective chiplet integration. In *Proceedings of the 2024 29th Asia and South Pacific Design Automation Conference*. IEEE, 58–64.
- [112] Tao Li, Jie Hou, Jinli Yan, Rulin Liu, Hui Yang, and Zhigang Sun. 2020. Chiplet heterogeneous integration technology—status and challenges. *Electronics* 9, 4 (2020), 670.
- [113] Xiao Li, Lin Chen, Shixi Chen, Fan Jiang, Chengeng Li, Wei Zhang, and Jiang Xu. 2024. Deep reinforcement learning-based power management for chiplet-based multicore systems. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32, 9 (2024), 1726–1739.
- [114] Zixi Li and David Wentzlaff. 2024. LUCIE: A universal chiplet-interposer design framework for plug-and-play integration. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture*. IEEE, 423–436.
- [115] Peiyu Liao, Yuxuan Zhao, Dawei Guo, Yibo Lin, and Bei Yu. 2023. Analytical die-to-die 3-D placement with bistratal wirelength model and GPU acceleration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 6 (2023), 1624–1637.
- [116] Bo-Qiao Lin, Ting-Chou Lin, and Yao-Wen Chang. 2016. Redistribution layer routing for integrated fan-out wafer-level chip-scale packages. In *Proceedings of the 2016 IEEE/ACM International Conference on Computer-Aided Design*. 1–8.
- [117] Mu-Shan Lin, Tze-Chiang Huang, Chien-Chun Tsai, King-Ho Tam, Kenny Cheng-Hsiang Hsieh, Ching-Fang Chen, Wen-Hung Huang, Chi-Wei Hu, Yu-Chi Chen, Sandeep Kumar Goel, et al. 2020. A 7-nm 4-GHz Arm¹-core-based CoWoS¹ chiplet design for high-performance computing. *IEEE Journal of Solid-State Circuits* 55, 4 (2020), 956–966.
- [118] Xinquan Lin, Haobo Xu, Yinhe Han, and Yiming Gan. 2024. HEX-SIM: Evaluating multi-modal large language models on multi-chiplet NPUs. In *Proceedings of the 2024 IEEE International Symposium on Workload Characterization*. IEEE, 108–120.
- [119] Feng Ling, Yufeng Dan, Changhua Wan, Kevin Cai, and Bidyut Sen. 2023. Machine-learning-based optimization of tx equalization parameters of a high-speed channel. In *Proceedings of the 2023 IEEE Symposium on Electromagnetic Compatibility and Signal/Power Integrity*. IEEE, 683–687.

- [120] Chester Liu, Jacob Botimer, and Zhengya Zhang. 2021. A 256Gb/s/mm-shoreline AIB-compatible 16nm FinFET CMOS chiplet for 2.5 D integration with stratix 10 FPGA on EMIB and tiling on silicon interposer. In *Proceedings of the 2021 IEEE Custom Integrated Circuits Conference*. IEEE, 1–2.
- [121] Chang Liu, Taigon Song, Jonghyun Cho, Joohee Kim, Joungho Kim, and Sung Kyu Lim. 2011. Full-chip TSV-to-TSV coupling analysis and optimization in 3D IC. In *Proceedings of the 48th Design Automation Conference*. 783–788.
- [122] Chang Liu, Taigon Song, and Sung Kyu Lim. 2011. Signal integrity analysis and optimization for 3D ICs. In *Proceedings of the 2011 12th International Symposium on Quality Electronic Design*. IEEE, 1–8.
- [123] Wen-Hao Liu, Min-Sheng Chang, and Ting-Chi Wang. 2014. Floorplanning and signal assignment for silicon interposer-based 3D ICs. In *Proceedings of the 51st Annual Design Automation Conference*. 1–6.
- [124] Xiaodong Liu, Yifan Zhang, Gary Yeap, and Xuan Zeng. 2011. An integrated algorithm for 3D-IC TSV assignment. In *Proceedings of the 48th Design Automation Conference*. 652–657.
- [125] Yafei Liu, Dejian Li, Zheng Yang, Chaoqin Zhang, Yunlai Zhang, Xiangyu Li, Mingwei Cao, and Shouyi Yin. 2025. A chiplet platform for intelligent radar/sonar leveraging domain-specific reusable active interposer. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33, 4 (2025), 903–915.
- [126] Gabriel H. Loh and Raja Swaminathan. 2023. The next era for chiplet innovation. In *Proceedings of the 2023 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1–6.
- [127] Gabriel H. Loh, Yuan Xie, and Bryan Black. 2007. Processor design in 3D die-stacking technologies. *IEEE Micro* 27, 3 (2007), 31–48.
- [128] Tiantao Lu, Caleb Serafy, Zhiyuan Yang, Sandeep Kumar Samal, Sung Kyu Lim, and Ankur Srivastava. 2017. TSV-based 3-D ICs: Design methods and tools. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 36, 10 (2017), 1593–1619.
- [129] Tiantao Lu and Ankur Srivastava. 2017. Low-power clock tree synthesis for 3D-ICs. *ACM Transactions on Design Automation of Electronic Systems* 22, 3 (2017), 1–24.
- [130] Yi-Chen Lu, Sai Surya Kiran Pentapati, Lingjun Zhu, Kambiz Samadi, and Sung Kyu Lim. 2020. TP-GNN: A graph neural network framework for tier partitioning in monolithic 3D ICs. In *Proceedings of the 2020 57th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [131] Hanzhi Ma, Da Li, Er-Ping Li, Andreas C. Cangellaris, and Xu Chen. 2022. A fast optimization method for high-speed link inverse design with SVR-AS algorithm. *IEEE Transactions on Signal and Power Integrity* 1 (2022), 22–31.
- [132] Xiaohan Ma, Ying Wang, Yujie Wang, Xuyi Cai, and Yinhe Han. 2022. Survey on chiplets: Interface, interconnect and integration methodology. *CCF Transactions on High Performance Computing* 4, 1 (2022), 43–52.
- [133] Yenai Ma, Leila Delshadtehrani, Cansu Demirkiran, José L. Abellan, and Aiav Joshi. 2021. TAP-2.5 D: A thermally-aware chiplet placement methodology for 2.5 D systems. In *Proceedings of the 2021 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1246–1251.
- [134] Pritam Majumder, Sungkeun Kim, Jiayi Huang, Ki Hwan Yum, and Eun Jung Kim. 2020. Remote control: A simple deadlock avoidance scheme for modular systems-on-chip. *IEEE Transactions on Computers* 70, 11 (2020), 1928–1941.
- [135] Alex Manukovsky, Yuriy Shlepnev, and Zurab Khasidashvili. 2021. Machine learning based design space exploration and applications to signal integrity analysis of 112Gb SerDes systems. In *Proceedings of the 2021 IEEE 71st Electronic Components and Technology Conference*. IEEE, 1234–1245.
- [136] Yafei Liu, Dejian Li, Zheng Yang, Chaoqin Zhang, Yunlai Zhang, Xiangyu Li, Mingwei Cao, and Shouyi Yin. 2025. A chiplet platform for intelligent radar/sonar leveraging domain-specific reusable active interposer. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33, 4 (2025), 903–915.
- [137] Larry McMurchie and Carl Ebeling. 1995. PathFinder: A negotiation-based performance-driven router for FPGAs. In *Proceedings of the 1995 ACM 3rd International Symposium on Field-programmable Gate Arrays*. 111–117.
- [138] Weiyang Miao, Zhen Xie, Chuan Seng Tan, and Mihai D. Rotaru. 2024. Deep reinforcement learning-based power distribution network design optimization for multi-chiplet system. In *Proceedings of the 2024 IEEE 74th Electronic Components and Technology Conference*. IEEE, 1716–1723.
- [139] Jacob Minz, Xin Zhao, and Sung Kyu Lim. 2008. Buffered clock tree synthesis for 3D ICs under thermal variations. In *Proceedings of the 2008 Asia and South Pacific Design Automation Conference*. IEEE, 504–509.
- [140] Michael Molter, Rahul Kumar, Sonja Koller, Osama Waqar Bhatti, Nikita Ambasana, Elyse Rosenbaum, and Madhavan Swaminathan. 2023. Thermal-aware SoC macro placement and multi-chip module design optimization with Bayesian optimization. In *Proceedings of the 2023 IEEE 73rd Electronic Components and Technology Conference*. IEEE, 935–942.
- [141] Gauthaman Murali, Aditya Iyer, Lingjun Zhu, Jianming Tong, Francisco Munoz Martinez, Srivatsa Rangachar Srinivasa, Tanay Karnik, Tushar Krishna, and Sung Kyu Lim. 2023. On continuing dnn accelerator architecture scaling using tightly coupled compute-on-memory 3-d ics. *IEEE Transactions on Very Large Scale Integration Systems* 31, 10 (2023), 1603–1613.

- [142] Gauthaman Murali, Heechun Park, Eric Qin, Hakki Mert Torun, Majid Ahadi Dolatsara, Madhavan Swaminathan, Tushar Krishna, and Sung Kyu Lim. 2021. Clock delivery network design and analysis for interposer-based 2.5-D heterogeneous systems. *IEEE Transactions on Very Large Scale Integration Systems* 29, 4 (2021), 605–616.
- [143] Samuel Naffziger, Noah Beck, Thomas Burd, Kevin Lepak, Gabriel H. Loh, Mahesh Subramony, and Sean White. 2021. Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product. In *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture*. IEEE, 57–70.
- [144] Pragnya Sudershan Nalla, Emad Haque, Yaotian Liu, Sachin S. Sapatnekar, Jeff Zhang, Chaitali Chakrabarti, and Yu Cao. 2025. CLAIRE: Composable chiplet libraries for AI inference. In *Proceedings of the 2025 Design, Automation and Test in Europe Conference*. IEEE, 1–7.
- [145] Jawad Nasrullah, Zhiquan Luo, and Greg Taylor. 2019. Designing software configurable chips and SIPs using chiplets and zGlue. In *Proceedings of the International Symposium on Microelectronics*. International Microelectronics Assembly and Packaging Society, 000027–000032.
- [146] Thong Nguyen, Tianjian Lu, Ken Wu, and Jose Schutt-Aine. 2019. Fast transient simulation of high-speed channels using recurrent neural network. *arXiv preprint arXiv:1902.02627* (2019).
- [147] August Ning, Georgios Tziantzioulis, and David Wentzlaff. 2023. Supply chain aware computer architecture. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [148] Mohanad Odema, Luke Chen, Hyoukjun Kwon, and Mohammad Abdullah Al Faruque. 2024. SCAR: Scheduling multi-model AI workloads on heterogeneous multi-chiplet module accelerators. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture*. IEEE, 565–579.
- [149] Deok Keun Oh, Mu Jun Choi, and Ju Ho Kim. 2019. Thermal-aware 3D symmetrical buffered clock tree synthesis. *ACM Transactions on Design Automation of Electronic Systems* 24, 3 (2019), 1–22.
- [150] Open Compute Project. 2023. Chiplet Data Exchange in XML Format (CDXML). Retrieved from <https://www.opencompute.org/chiplets/31/chiplet-data-exchange-in-xml-format-cdxml> (2023).
- [151] Sergii Osmolovskiy, Johann Knechtel, Igor L. Markov, and Jens Lienig. 2018. Optimal die placement for interposer-based 3D ICs. In *Proceedings of the 2018 23rd Asia and South Pacific Design Automation Conference*. IEEE, 513–520.
- [152] Jiwoo Pak, Mohit Pathak, Sung Kyu Lim, and David Z. Pan. 2011. Modeling of electromigration in through-silicon-via based 3D IC. In *Proceedings of the 2011 IEEE 61st Electronic Components and Technology Conference*. IEEE, 1420–1427.
- [153] Saptadeep Pal and Puneet Gupta. 2020. Pathfinding for 2.5 D interconnect technologies. In *Proceedings of the Workshop on System-Level Interconnect: Problems and Pathfinding Workshop*. 1–8.
- [154] Saptadeep Pal, Daniel Petrisko, Rakesh Kumar, and Puneet Gupta. 2020. Design space exploration for chiplet-assembly-based 3D ICs. In *Proceedings of the 2018 23rd Asia and South Pacific Design Automation Conference*. IEEE, 1062–1073.
- [155] Vasil Pano, Ragh Kuttappa, and Baris Taskin. 2019. 3D NoCs with active interposer for multi-die systems. In *Proceedings of the 13th IEEE/ACM International Symposium on Networks-on-Chip*. 1–8.
- [156] Nicolas Pantano, Cesar Roda Neve, Geert Van der Plas, Mikael Detalle, Marian Verhelst, Marc Heyns, and Eric Beyne. 2016. Technology optimization for high bandwidth density applications on 3D interposer. In *Proceedings of the 2016 6th Electronic System-Integration Technology Conference*. IEEE, 1–6.
- [157] Heechun Park, Jinwoo Kim, Venkata Chaitanya Krishna Chekuri, Majid Ahadi Dolatsara, Mohammed Nabeel, Alabi Bojesomo, Satwik Patnaik, Ozgur Sinanoglu, Madhavan Swaminathan, Saibal Mukhopadhyay, et al. 2020. Design flow for active interposer-based 2.5-D ICs and study of RISC-V architecture with secure NoC. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 10, 12 (2020), 2047–2060.
- [158] Hyunwook Park, Minsu Kim, Seongguk Kim, Keunwoo Kim, Haeyeon Kim, Taein Shin, Keeyoung Son, Boogyo Sim, Subin Kim, Seungtaek Jeong, et al. 2022. Transformer network-based reinforcement learning method for power distribution network (PDN) optimization of high bandwidth memory (HBM). *IEEE Transactions on Microwave Theory and Techniques* 70, 11 (2022), 4772–4786.
- [159] Hyunwook Park, Junyong Park, Subin Kim, Kyungjun Cho, Daehwan Lho, Seungtaek Jeong, Shinyoung Park, Gapyeol Park, Boogyo Sim, Seongguk Kim, et al. 2020. Deep reinforcement learning-based optimal decoupling capacitor design method for silicon interposer-based 2.5-D/3-D ICs. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 10, 3 (2020), 467–478.
- [160] Jaehyun Park, Alish Kanani, Lukas Pfromm, Harsh Sharma, Parth Solanki, Eric Tervo, Janardhan Rao Doppa, Partha Pratim Pande, and Umit Y. Ogras. 2024. Thermal modeling and management challenges in heterogenous integration: 2.5 D chiplet platforms and beyond. In *Proceedings of the 2024 IEEE 42nd VLSI Test Symposium*. IEEE, 1–4.
- [161] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32 (2019), 8026–8037.
- [162] Mohit Pathak, Young-Joon Lee, Thomas Moon, and Sung Kyu Lim. 2010. Through-silicon-via management during 3D physical design: When to add and how many?. In *Proceedings of the 2010 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 387–394.

- [163] Christos Pavlidis, Nikos Sketopoulos, Christos P. Sotiriou, and Vasilis F. Pavlidis. 2024. Ternary clock tree synthesis for 3D ICs. In *Proceedings of the 2024 20th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design*. IEEE, 1–4.
- [164] Lukas Pfromm, Alish Kanani, Harsh Sharma, Parth Solanki, Eric Tervo, Jaehyun Park, Janardhan Rao Doppa, Partha Pratim Pande, and Umit Y. Ogras. 2024. MFIT: Multi-fidelity thermal modeling for 2.5 D and 3D multi-chiplet architectures. arXiv:2410.09188. Retrieved from <https://arxiv.org/abs/2410.09188>
- [165] Bahaa Radi, Zonghao Li, Dhruv Patel, and Anthony Chan Carusone. 2023. Optimizing the photodetector/analog front-end interface in optical communication receivers. *IEEE Transactions on Signal and Power Integrity 2* (2023), 111–121.
- [166] Riko Radojcic. 2017. *More-than-Moore 2.5 D and 3D SiP Integration*. Springer.
- [167] Chong Ran, Qiang Zeng, and Yuanqing Cheng. 2023. A hierarchical clock network synthesis method for 3d integrated circuits with obstacle avoidance. In *Proceedings of the 2023 International Symposium of Electronics Design Automation*. IEEE, 345–350.
- [168] Cristhian Roman-Vicharra, Yiran Chen, and Jiang Hu. 2025. PPAC driven multi-die and multi-technology floorplanning. Retrieved from <https://arxiv.org/abs/2502.10932>
- [169] Yousef Safari, Pooya Aghanoury, Subramanian S. Iyer, Nader Sehatbakhsh, and Boris Vaisband. 2023. Hybrid obfuscation of chiplet-based systems. In *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [170] Alberto Sangiovanni-Vincentelli, Zheng Liang, Zhe Zhou, and Jiayi Zhang. 2023. Automated design of chiplets. In *Proceedings of the 2023 International Symposium on Physical Design*. 1–8.
- [171] Daniel P. Seemuth, Azadeh Davoodi, and Katherine Morrow. 2015. Automatic die placement and flexible I/O assignment in 2.5 D IC design. In *Proceedings of the 16th International Symposium on Quality Electronic Design*. IEEE, 524–527.
- [172] Jaime Sevilla, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. 2022. Compute trends across three eras of machine learning. In *Proceedings of the 2022 International Joint Conference on Neural Networks*. IEEE, 1–8.
- [173] Debendra Das Sharma. 2023. Compute Express Link™(CXL™): An Open Interconnect for Cloud Infrastructure. In *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference*. IEEE, 1–4.
- [174] Debendra Das Sharma, Gerald Pasdast, Zhiguo Qian, and Kemal Aygun. 2022. Universal chiplet interconnect express (UCIe): An open industry standard for innovations with chiplets at package level. *IEEE Transactions on Components, Packaging and Manufacturing Technology 12*, 9 (2022), 1423–1431.
- [175] Harsh Sharma, Sumit K. Mandal, Janardhan Rao Doppa, Umit Y. Ogras, and Partha Pratim Pande. 2022. SWAP: A server-scale communication-aware chiplet-based manycore PIM accelerator. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 41*, 11 (2022), 4145–4156.
- [176] Harsh Sharma, Lukas Pfromm, Rasit Onur Topaloglu, Janardhan Rao Doppa, Umit Y. Ogras, Ananth Kalyanraman, and Partha Pratim Pande. 2023. Florets for chiplets: Data flow-aware high-performance and energy-efficient network-on-interposer for CNN inference tasks. *ACM Transactions on Embedded Computing Systems 22*, 5s (2023), 1–21.
- [177] Yujeong Shim, Woon Seong Kwon, and Anna Qao. 2022. Design and Technology Spaces for Heterogeneous Chiplet Integration. Retrieved February 21, 2026 from <https://designthesolution.org/wp-content/uploads/2022/09/Design-and-technology-spaces-for-heterogeneous-chiplet-integration.pdf>
- [178] Prachi Shukla, Derrick Aguren, Tom Burd, Ayse K. Coskun, and John Kalamatianos. 2023. Temperature-aware sizing of multi-chip module accelerators for multi-DNN workloads. In *Proceedings of the 2023 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1–6.
- [179] Alan Smith and Vamsi Krishna Alla. 2025. AMD instinct MI300X: A generative AI accelerator and platform architecture. *IEEE Micro 45*, 3 (2025), 41–48.
- [180] Alan Smith, Gabriel H. Loh, Samuel Naffziger, John Wu, Nathan Kalyanasundharam, Eric Chapman, Raja Swaminathan, Tyrone Huang, Wonjun Jung, Alexander Kaganov, et al. 2024. Interconnect design for heterogeneous integration of chiplets in the AMD instinct MI300X accelerator. *IEEE Micro 45*, 1 (2024), 57–66.
- [181] Alan Smith, Gabriel H. Loh, Michael J. Schulte, Mike Ignatowski, Samuel Naffziger, Mike Mantor, Mark Fowler Nathan Kalyanasundharam, Vamsi Alla, Nicholas Malaya, Joseph L. Greathouse, et al. 2024. Realizing the AMD exascale heterogeneous processor vision: Industry product. In *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture*. IEEE, 876–889.
- [182] Atefeh Sohrabizadeh, Cody Hao Yu, Min Gao, and Jason Cong. 2022. AutoDSE: Enabling software programmers to design efficient FPGA accelerators. *ACM Transactions on Design Automation of Electronic Systems 27*, 4 (2022), 1–27.
- [183] Srivatsa Rangachar Srinivasa, Dileep Kurian, Paolo Aseron, Prerna Budhkar, Arun Radhakrishnan, Alejandro Cardenas Lopez, Jainaveen Sundaram, Vinayak Honkote, Leonid Azarenkov, Daniel Lake, et al. 2025. A 300MB SRAM, 20Tb/s bandwidth scalable heterogenous 2.5 D system inferencing simultaneous streams across 20 chiplets with workload-dependent configurations. In *Proceedings of the 2025 IEEE International Solid-State Circuits Conference*. IEEE, 50–52.

- [184] Srivatsa Rangachar Srinivasa, Jainaveen Sundaram Priya, Dileep Kurian, Erika Ramirez Lozano, Satish Yada, Saransh Chhabra, Kamakhya Prasad Sahu, Paolo Aseron, Ronald Kalim, Anuradha Srinivasan, et al. 2022. Design methodology for scalable 2.5 D/3D heterogenous tiled chiplet systems. In *Proceedings of the 2022 23rd International Symposium on Quality Electronic Design*. IEEE, 1–4.
- [185] Dylan Stow, Itir Akgun, Russell Barnes, Peng Gu, and Yuan Xie. 2016. Cost analysis and cost-driven IP reuse methodology for SoC design based on 2.5 D/3D integration. In *Proceedings of the 2016 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 1–6.
- [186] Dylan Stow, Itir Akgun, Russell Barnes, Peng Gu, and Yuan Xie. 2016. Cost and thermal analysis of high-performance 2.5 D and 3D integrated circuit design space. In *Proceedings of the 2016 IEEE Computer Society Annual Symposium on VLSI*. IEEE, 637–642.
- [187] Dylan Stow, Itir Akgun, and Yuan Xie. 2019. Investigation of cost-optimal network-on-chip for passive and active interposer systems. In *Proceedings of the 2019 ACM/IEEE International Workshop on System Level Interconnect Prediction*. IEEE, 1–8.
- [188] Chetan Choppali Sudarshan, Nikhil Matkar, Sarma Vrudhula, Sachin S. Sapatnekar, and Vidya A. Chhabria. 2024. Eco-chip: Estimation of carbon footprint of chiplet-based architectures for sustainable vlsi. In *Proceedings of the 2024 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 671–685.
- [189] H. Ekin Sumbul, Arne Symons, Lita Yang, Huichu Liu, Tony F. Wu, Matheus Trevisan Moreira, Debabrata Mohapatra, Abhinav Agarwal, Kaushik Ravindran, Chris Thompson, et al. 2025. A 3D design methodology for integrated wearable SoCs: Enabling energy efficiency and enhanced performance at Iso-area footprint. In *Proceedings of the 2025 Design, Automation and Test in Europe Conference*. IEEE, 1–7.
- [190] Synopsys. 2023. How Quickly will Multi-Die Systems Change Semiconductor Design? Retrieved February 21, 2026 from <https://www.synopsys.com/content/dam/synopsys/solutions/multi-die/white-papers/synopsys-multi-die-%0A%0A%0Asystem-solution-wp.pdf>
- [191] Ebadollah Taheri, Sudeep Pasricha, and Mahdi Nikdast. 2022. DeFT: A deadlock-free and fault-tolerant routing algorithm for 2.5 D chiplet networks. In *Proceedings of the 2022 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1047–1052.
- [192] Emil Talpes, Douglas Williams, and Debjit Das Sarma. 2022. Dojo: The microarchitecture of tesla’s exa-scale computer. In *Proceedings of the 2022 IEEE Hot Chips 34 Symposium*. IEEE Computer Society, 1–28.
- [193] Zhanhong Tan, Hongyu Cai, Runpei Dong, and Kaisheng Ma. 2021. NN-baton: DNN workload orchestration and chiplet granularity exploration for multichip accelerators. In *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture*. IEEE, 1013–1026.
- [194] Jinwei Tang, Jiayin Qin, Nuo Xu, Pragnya Sudershan Nalla, Yu Cao, Yang Katie Zhao, and Caiwen Ding. 2025. MAHL: Multi-a agent LLM-guided hierarchical chiplet design with adaptive debugging. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [195] Aida Todri-Sanial, Sandip Kundu, Patrick Girard, Alberto Bosio, Luigi Dilillo, and Arnaud Virazel. 2013. Globally constrained locally optimized 3-D power delivery networks. *IEEE Transactions on very large scale Integration Systems* 22, 10 (2013), 2131–2144.
- [196] Xingyu Tong, Yuhao Ren, Zhijie Cai, Peng Zou, Min Wei, Yuan Wen, Zhifeng Lin, and Jianli Chen. 2025. OO: Optimized one-die placement for face-to-face bonded 3D ICs. *Integration* 102 (2025), 102371.
- [197] Rao R. Tummala. 2005. Packaging: past, present and future. In *Proceedings of the 2005 6th International Conference on Electronic Packaging Technology*. IEEE, 3–7.
- [198] Pruek Vanna-Iampikul, Chengjia Shao, Yi-Chen Lu, Sai Pentapati, and Sung Kyu Lim. 2021. Snap-3D: A constrained placement-driven physical design methodology for face-to-face-bonded 3D ICs. In *Proceedings of the 2021 International Symposium on Physical Design*. 39–46.
- [199] E. Jan Vardaman. 2021. Too Many Package Options: What Makes Sense for Your Application? Retrieved February 21, 2026 from https://hc33.hotchips.org/assets/program/tutorials/TechSearchInternational_TutorialHotChips%20FINAL.pdf
- [200] Mengdi Wang, Ying Wang, Cheng Liu, and Lei Zhang. 2021. Network-on-interposer design for agile neural-network processor chip customization. In *Proceedings of the 2021 58th ACM/IEEE Design Automation Conference*. IEEE, 49–54.
- [201] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. 2020. Deep graph library: A graph-centric, highly-performant package for graph neural networks. (2020). Retrieved from <https://arxiv.org/abs/1909.01315>
- [202] Qipan Wang, Xueqing Li, Tianyu Jia, Yibo Lin, Runsheng Wang, and Ru Huang. 2024. ATPlace2. 5D: Analytical thermal-aware chiplet placement framework for large-scale 2.5 D-IC. In *Proceedings of the 2024 International Conference on Computer-Aided Design*.
- [203] Shengcheng Wang, Farshad Firouzi, Fabian Oboril, and Mehdi B. Tahoori. 2014. P/G TSV planning for IR-drop reduction in 3D-ICs. In *Proceedings of the 2014 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, 1–6.

- [204] Shengcheng Wang, Zeyu Sun, Yuan Cheng, Sheldon X-D Tan, and Mehdi B. Tahoori. 2017. Leveraging recovery effect to reduce electromigration degradation in power/ground TSV. In *Proceedings of the 2017 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 811–818.
- [205] Xiaohang Wang, Yifan Wang, Yingtao Jiang, Amit Kumar Singh, and Mei Yang. 2024. On task mapping in multi-chiplet based many-core systems to optimize inter-and intra-chiplet communications. *IEEE Transactions on Computers* 74, 2 (2024), 510–525.
- [206] Xiaohang Wang, Miao Xu, Amit Kumar Singh, Yingtao Jiang, and Mei Yang. 2025. On optimizing inter-and intra-chiplet interconnection topologies for robust multi-chiplet systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 10 (2025), 3976–3989.
- [207] Yujie Wang, Pu Chen, Jiang Hu, and Jeyavijayan Rajendran. 2016. The cat and mouse in split manufacturing. In *Proceedings of the 53rd Annual Design Automation Conference*. 1–6.
- [208] Zhenyu Wang, Pragnya Sudershan Nalla, Jingbo Sun, A. Alper Goksoy, Sumit K. Mandal, Jae-sun Seo, Vidya A. Chhabria, Jeff Zhang, Chaitali Chakrabarti, Umit Y. Ogras, et al. 2025. HISIM: Analytical performance modeling and design space exploration of 2.5 D/3D integration for AI computing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 8 (2025), 3208–3221.
- [209] Zhimin Wang, Guigen Ye, Xinjian Li, Shifeng Xue, and Liang Gong. 2021. Thermal–mechanical performance analysis and structure optimization of the TSV in 3-D IC. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 11, 5 (2021), 822–831.
- [210] Hsiang-Ting Wen, Yu-Jie Cai, Yang Hsu, and Yao-Wen Chang. 2022. Via-based redistribution layer routing for InFO packages with irregular pad structures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 12 (2022), 5554–5567.
- [211] Seungmin Woo, Pruek Vanna-iampikul, and Sung Kyu Lim. 2024. AI-driven evaluation and optimization of bump pitch effects on chiplet and interposer design quality. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [212] Yibo Wu, Liang Wang, Xiaohang Wang, Jie Han, Jianfeng Zhu, Honglan Jiang, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2022. Upward packet popup for deadlock freedom in modular chiplet-based systems. In *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 986–1000.
- [213] John Wu, Rahul Agarwal, Michael Ciraula, Carl Dietz, Brett Johnson, Dave Johnson, Russell Schreiber, Raja Swaminathan, Will Walker, and Samuel Naffziger. 2022. 3D V-Cache: The implementation of a hybrid-bonded 64MB stacked cache for a 7nm x86-64 CPU. In *Proceedings of the 2022 IEEE International Solid-State Circuits Conference*. IEEE, 428–429.
- [214] Jing Xia, Chuanning Cheng, Xiping Zhou, Yuxing Hu, and Peter Chun. 2021. Kunpeng 920: The first 7-nm chiplet-based 64-core arm soc for cloud services. *IEEE Micro* 41, 5 (2021), 67–75.
- [215] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, et al. 2022. Towards developing high performance RISC-V processors using agile methodology. In *Proceedings of the 2022 55th IEEE/ACM International Symposium on Microarchitecture*. IEEE, 1178–1199.
- [216] Yaoyao Ye, Zixuan Liu, Jungan Liu, and Li Jiang. 2023. ASDR: An application-specific deadlock-free routing for chiplet-based systems. In *Proceedings of the 16th International Workshop on Network on Chip Architectures*. 46–51.
- [217] Jieming Yin, Zhifeng Lin, Onur Kayiran, Matthew Poremba, Muhammad Shoaib Bin Altaf, Natalie Enright Jerger, and Gabriel H. Loh. 2018. Modular routing design for chiplet-based systems. In *Proceedings of the 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture*. IEEE, 726–738.
- [218] Jinming Zhang, Xi Fan, Yaoyao Ye, Xuyan Wang, Guojie Xiong, Xianglun Leng, Ningyi Xu, Yong Lian, and Guanghui He. 2023. INDM: Chiplet-based interconnect network and dataflow mapping for DNN accelerators. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 4 (2023), 1107–1120.
- [219] Jie Zhang, Wei Lu, Po-Tsang Huang, Sih-Han Li, Tsung-Yi Hung, Shih-Hsien Wu, Ming-Ji Dai, I-Shan Chung, Wen-Chao Chen, Chin-Hung Wang, et al. 2022. An embedded multi-die active bridge (EMAB) chip for rapid-prototype programmable 2.5 D/3D packaging technology. In *Proceedings of the 2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 262–263.
- [220] Jinming Zhang, Xuyan Wang, Yaoyao Ye, Dongxu Lyu, Guojie Xiong, Ningyi Xu, Yong Lian, and Guanghui He. 2024. M2M: A fine-grained mapping framework to accelerate multiple dnns on a multi-chiplet architecture. *IEEE Transactions on Very Large Scale Integration Systems* 32, 10 (2024), 1864–1877.
- [221] Peng Zhang, Da-Wei Wang, and Wen-Sheng Zhao. 2024. A thermal and power integrity co-optimization framework for 2.5-D integrated microsystem. *IEEE Transactions on Circuits and Systems I: Regular Papers* 72, 3 (2024), 1397–1410.
- [222] Jiechen Zhao, Natalie Enright Jerger, and Mingyu Gao. 2021. What can chiplets bring to multi-tenant clouds. In *Proceedings of the Cloud Workshop at Proceedings of the 54th IEEE/ACM International Symposium on Microarchitecture*.
- [223] Yujie Zhao, Yang Zhao, Cheng Wan, and Yingyan Lin. 2024. 3D-Carbon: An analytical carbon modeling tool for 3D and 2.5 D integrated circuits. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.

- [224] Yuxuan Zhao, Lancheng Zou, and Bei Yu. 2025. Physical design for advanced 3D ICs: Challenges and solutions. In *Proceedings of the International Symposium on Physical Design*.
- [225] Hao Zheng, Ke Wang, and Ahmed Louri. 2020. A versatile and flexible chiplet-based system design for heterogeneous manycore architectures. In *Proceedings of the 2020 57th ACM/IEEE Design Automation Conference*. IEEE, 1–6.
- [226] Haocong Zhi, Xianuo Xu, Weijian Han, Zhilin Gao, Xiaohang Wang, Maurizio Palesi, Amit Kumar Singh, and Letian Huang. 2021. A methodology for simulating multi-chiplet systems using open-source simulators. In *Proceedings of the 8th Annual ACM International Conference on Nanoscale Computing and Communication*. 1–6.
- [227] Jun Zhou, Huawei Li, Yuntan Fang, Tiancheng Wang, Yuanqing Cheng, and Xiaowei Li. 2014. HARS: A high-performance reliable routing scheme for 3D NoCs. In *Proceedings of the 2014 IEEE Computer Society Annual Symposium on VLSI*. IEEE, 392–397.
- [228] Lingjun Zhu, Jiawei Hu, Gauthaman Murali, and Sung Kyu Lim. 2024. Hetero-3d: Maximizing performance and power delivery benefits of heterogeneous 3d ics. In *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design*. 1–6.
- [229] Ting Zhu, C. Yongjin, C. Jacky, and C. Chris. 2019. Accelerating 56G PAM4 link equalization optimization using machine learning-based analysis. In *Proceedings of the Semicond. Electron. Des. Conf.*
- [230] Zhen Zhuang, Weishiun Hung, Md Arafat Kabir, Yarui Peng, and Tsung-Yi Ho. 2026. Adaptive redistribution layer routing for chiplet-package co-design in 2.5D system. *ACM Transactions on Design Automation of Electronic Systems* 31, 3 (2026).
- [231] Zhen Zhuang, Bei Yu, Kai-Yuan Chao, and Tsung-Yi Ho. 2022. Multi-package co-design for chiplet integration. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.

Received 25 July 2025; revised 24 January 2026; accepted 1 February 2026